LAPORAN PRAKTIKUM

PEMROGRAMAN WEB – GIK1JAB3

MODUL 9: CODEIGNITER 3 + HMVC + REST API + CRUD AJAX TANPA REFRESH

Alya Mutiya Ningsih Pertiwi

707012500132

D4 SIKC 49-04

PROGRAM STUDI D4 SISTEM INFORMASI KOTA CERDAS

FAKULTAS ILMU TERAPAN

UNIVERSITAS TELKOM

BANDUNG

2026

1. Tujuan Praktikum

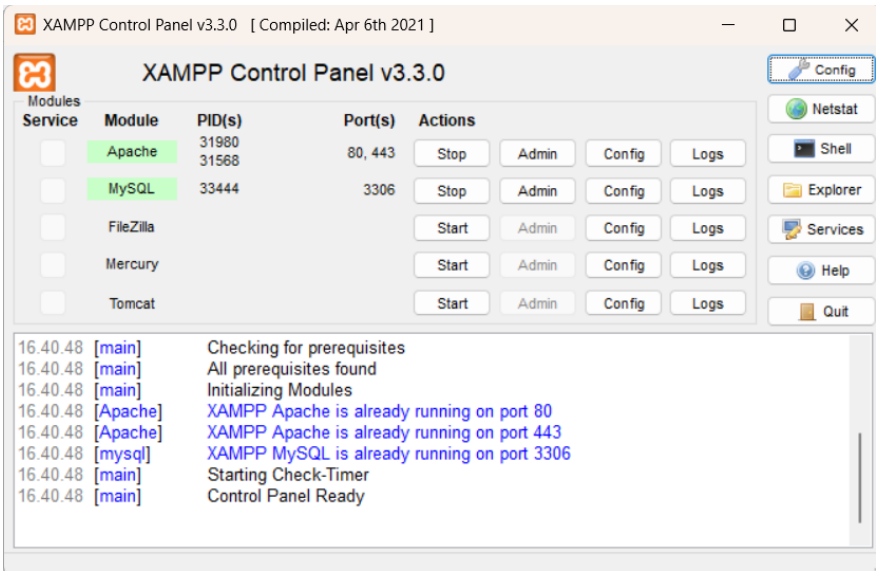
1. Memahami konsep HMVC pada CodeIgniter 3.
2. Mengimplementasikan struktur modular (feature-based).
3. Membuat REST API berbasis JSON.
4. Mengintegrasikan Model dengan REST Controller.
5. Menggunakan AJAX untuk komunikasi asynchronous.
6. Membuat fitur CRUD tanpa refresh halaman.
7. Menguji API menggunakan Postman.
8. Mengintegrasikan frontend dengan REST API.

2. Alat dan Bahan

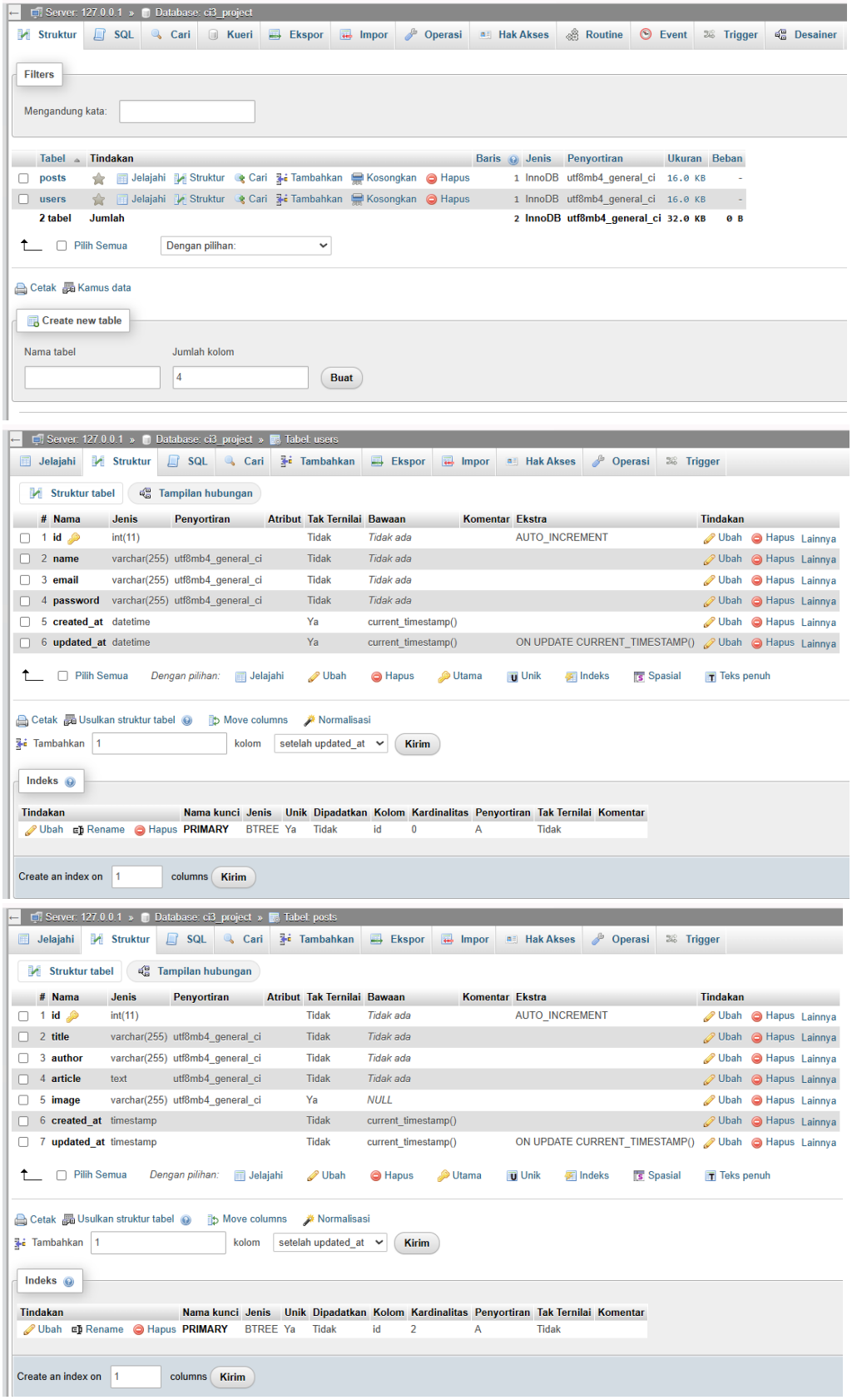
1. XAMPP (Apache & MySQL aktif)
2. PHP 7.x
3. CodeIgniter 3
4. Library HMVC (Modular Extensions – MX)
5. Library REST Server (chriskacerguis)
6. Postman
7. VS Code / Code Editor
8. Browser (Chrome/Firefox)

3. Langkah-langkah Praktikum

1. Buka Aplikasi XAMPP lalu aktifkan menu “start” pada Apache dan MySQL.



2. Masuk pada bagian “Admin” dalam XAMPP pada MySQL, kemudian persiapkan database.



3. Masuk ke bagian “Explorer” di pojok kanan, lalu masuk ke dalam file bernama “htdocs”, kemudian buat folder baru dan pindahkan folder Code Igniter yang telah di buat pada Tugas latihan Pekan-8 pada folder baru tersebut, lalu buat program php pada folder tersebut pada Visual Studio Code.

4. Mengubah code pada application/config/database.php untuk membuat koneksi antara CI3 dengan database MySQLmenggunakan Visual Studio Code.

```
$active_group = 'default';
$query_builder = TRUE;

$db['default'] = array(
    'dsn'   => '',
    'hostname' => 'localhost',
    'username' => 'root',
    'password' => '',
    'database' => 'ci3_project',
    'dbdriver' => 'mysqli',
    'dbprefix' => '',
    'pconnect' => FALSE,
    'db_debug' => (ENVIRONMENT !== 'productio
n'),'cache_on' => FALSE,
    'cachedir' => '',
    'char_set' => 'utf8',
    'dbcollat' => 'utf8_general_ci',
    'swap_pre' => '',
    'encrypt' => FALSE,
    'compress' => FALSE,
    'stricton' => FALSE,
    'failover' => array(),
    'save_queries' => TRUE
);
```

5. Tambahkan struktur HMVC pada folder application/modules. Buat folder Crudjs untuk fitur CRUD AJAX di dalam modules, kemudian buat folder controllers lalu pada Folder Controllers buat file Crudjs.php. Isi program Crudjs.php:

```php
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Crudjs extends CI_Controller {

    public function __construct() {
        parent::__construct();
        $this->load->model('Crudjs_model');
        $this->load->helper(array('url', 'form'));
        $this->load->library(array('form_validation', 'upload', 'session'));
    }

    public function debug() {
        $debug_info = array();

        $debug_info['database'] = array(
            'connected' => $this->db->conn_id ? 'Yes' : 'No',
            'database' => $this->db->database
        );

        $debug_info['table_exists'] = $this->db->table_exists('posts') ? 'Yes' : 'No';

        $query = $this->db->get('posts', 1);
        $debug_info['table_accessible'] = $query ? 'Yes' : 'No';
        $debug_info['record_count'] = $this->db->count_all('posts');

        $upload_path = './uploads/posts/';
        $debug_info['upload_folder'] = array(
            'exists' => is_dir($upload_path) ? 'Yes' : 'No',
            'writable' => is_writable($upload_path) ? 'Yes' : 'No'
        );

        $debug_info['sample_records'] = array();
        try {
            $records = $this->Crudjs_model->get_all();
            if (!empty($records)) {
                // Show first record as sample
                $first = $records[0];
                $debug_info['sample_records'] = array(
                    'count' => count($records),
                    'first_record' => $first
                );
            }
        } catch (Exception $e) {
            $debug_info['sample_records']['error'] = $e->getMessage();
        }

        echo '<pre>' . json_encode($debug_info, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES) . '</pre
>'; }

    public function index() {
        $data['title'] = 'CRUD With AJAX';
        $this->load->view('crudjs/index', $data);
    }

    public function get_all() {
        ob_clean();
        header('Content-Type: application/json; charset=utf-8');

        try {
            $records = $this->Crudjs_model->get_all();

            $response = array(
                'status' => 'success',
                'data' => $records
            );

            echo json_encode($response, JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        } catch (Exception $e) {
            echo json_encode(array(
                'status' => 'error',
                'message' => 'Failed to load records: ' . $e->getMessage()
            ));
        }
        exit;
    }

    public function get_record($id) {
        ob_clean();
        header('Content-Type: application/json; charset=utf-8');

        try {
            if (!$this->Crudjs_model->exists($id)) {
                echo json_encode(array(
                    'status' => 'error',
                    'message' => 'Record not found'
                ));
                exit;
            }

            $record = $this->Crudjs_model->get_by_id($id);

            echo json_encode(array(
                'status' => 'success',
                'data' => $record
            ), JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        } catch (Exception $e) {
            echo json_encode(array(
                'status' => 'error',
                'message' => 'Failed to load record: ' . $e->getMessage()
            ));
        }
        exit;
    }

    public function store() {
        ob_clean();
        header('Content-Type: application/json; charset=utf-8');

        try {
            $this->form_validation->set_rules('title', 'Title', 'required|max_length[255]');
            $this->form_validation->set_rules('author', 'Author', 'required|max_length[255]');
            $this->form_validation->set_rules('article', 'Article', 'required');

            if ($this->form_validation->run() === FALSE) {
                echo json_encode(array(
                    'status' => 'error',
                    'message' => 'Validation error: ' . strip_tags(validation_errors())
                ));
                exit;
            }

            $data = array(
                'title' => $this->input->post('title'),
                'author' => $this->input->post('author'),
                'article' => $this->input->post('article'),
                'created_at' => date('Y-m-d H:i:s'),
                'updated_at' => date('Y-m-d H:i:s')
            );

            if (!empty($_FILES['image']['name'])) {
                $upload_result = $this->_upload_image();

                if ($upload_result['status']) {
                    $data['image'] = 'posts/' . $upload_result['file_name'];
                } else {
                    echo json_encode(array(
                        'status' => 'error',
                        'message' => $upload_result['error']
                    ));
                    exit;
                }
            }

            $record_id = $this->Crudjs_model->insert($data);

            if ($record_id) {
                $new_record = $this->Crudjs_model->get_by_id($record_id);
                echo json_encode(array(
                    'status' => 'success',
                    'message' => 'Record created successfully!',
                    'data' => $new_record
                ), JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            } else {
                echo json_encode(array(
                    'status' => 'error',
                    'message' => 'Failed to create record'
                ));
            }
        } catch (Exception $e) {
            echo json_encode(array(
                'status' => 'error',
                'message' => 'Exception error: ' . $e->getMessage()
            ));
        }
        exit;
    }
```

```php
    public function update($id) {
        ob_clean();
        header('Content-Type: application/json; charset=utf-8');

        try {
            if (!$this->Crudjs_model->exists($id)) {
                echo json_encode(array(
                    'status' => 'error',
                    'message' => 'Record not found'
                ));
                exit;
            }

            $this->form_validation->set_rules('title', 'Title', 'max_length[255]');
            $this->form_validation->set_rules('author', 'Author', 'max_length[255]');

            if ($this->form_validation->run() === FALSE) {
                echo json_encode(array(
                    'status' => 'error',
                    'message' => 'Validation error: ' . strip_tags(validation_errors
())
                ));
                exit;
            }

            $data = array(
                'updated_at' => date('Y-m-d H:i:s')
            );

            if ($this->input->post('title')) {
                $data['title'] = $this->input->post('title');
            }
            if ($this->input->post('author')) {
                $data['author'] = $this->input->post('author');
            }
            if ($this->input->post('article')) {
                $data['article'] = $this->input->post('article');
            }

            if (!empty($_FILES['image']['name'])) {
                $old_record = $this->Crudjs_model->get_by_id($id);
                if ($old_record->image) {
                    $old_image_path = './uploads/' . $old_record->image;
                    if (file_exists($old_image_path)) {
                        unlink($old_image_path);
                    }
                }

                $upload_result = $this->_upload_image();

                if ($upload_result['status']) {
                    $data['image'] = 'posts/' . $upload_result['file_name'];
                } else {
                    echo json_encode(array(
                        'status' => 'error',
                        'message' => $upload_result['error']
                    ));
                    exit;
                }
            }

            $result = $this->Crudjs_model->update($id, $data);

            if ($result) {
                $updated_record = $this->Crudjs_model->get_by_id($id);
                echo json_encode(array(
                    'status' => 'success',
                    'message' => 'Record updated successfully!',
                    'data' => $updated_record
                ), JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            } else {
                echo json_encode(array(
                    'status' => 'error',
                    'message' => 'Failed to update record'
                ));
            }
        } catch (Exception $e) {
            echo json_encode(array(
                'status' => 'error',
                'message' => 'Exception error: ' . $e->getMessage()
            ));
        }
        exit;
    }

    public function delete($id) {
        ob_clean();
        header('Content-Type: application/json; charset=utf-8');

        try {
            $record = $this->Crudjs_model->get_by_id($id);

            if (!$record) {
                echo json_encode(array(
                    'status' => 'error',
                    'message' => 'Record not found'
                ));
                exit;
            }

            if ($record->image) {
                $image_path = './uploads/' . $record->image;
                if (file_exists($image_path)) {
                    unlink($image_path);
                }
            }

            $result = $this->Crudjs_model->delete($id);

            if ($result) {
                echo json_encode(array(
                    'status' => 'success',
                    'message' => 'Record deleted successfully!'
                ));
            } else {
                echo json_encode(array(
                    'status' => 'error',
                    'message' => 'Failed to delete record'
                ));
            }
        } catch (Exception $e) {
            echo json_encode(array(
                'status' => 'error',
                'message' => 'Exception error: ' . $e->getMessage()
            ));
        }
        exit;
    }

    public function test() {
        ob_clean();
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode(array('status' => 'success', 'message' => 'API is working'));
        exit;
    }

    private function _upload_image() {
        $upload_path = './uploads/posts/';

        if (!is_dir($upload_path)) {
            mkdir($upload_path, 0777, true);
        }

        $original_name = $_FILES['image']['name'];
        $sanitized_name = preg_replace('/[^A-Za-z0-9._-]/', '_', $original_name);
        $file_name = time() . '_' . $sanitized_name;

        $config['upload_path']      = $upload_path;
        $config['allowed_types']    = 'jpg|jpeg|png';
        $config['max_size']         = 2048; // 2MB
        $config['file_name']        = $file_name;
        $config['overwrite']        = FALSE;

        $this->upload->initialize($config);

        if ($this->upload->do_upload('image')) {
            $upload_data = $this->upload->data();
            return array(
                'status' => TRUE,
                'file_name' => $upload_data['file_name']
            );
        } else {
            return array(
                'status' => FALSE,
                'error' => $this->upload->display_errors('', '')
            );
        }
    }
}
```

6. Tambahkan folder baru bernama models pada application\modules\Crudjs kemudian buat file baru bernama Crudjs_model.php. Isi program Crudjs_model.php:

```php
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Crudjs_model extends CI_Model {

    private $table = 'posts';

    public function __construct() {
        parent::__construct();
        $this->load->database();
    }

    public function get_all() {
        $query = $this->db->get($this->table);
        $records = $query->result();

        foreach ($records as $record) {
            if ($record->image) {
                $record->image_url = 'http://localhost/ci3_project/uploads/posts/' . $record->imag
e;          } else {
                $record->image_url = null;
            }
        }

        return $records;
    }

    public function get_by_id($id) {
        $query = $this->db->get_where($this->table, array('id' => $id));
        $record = $query->row();

        if ($record && $record->image) {
            $record->image_url = 'http://localhost/ci3_project/uploads/posts/' . $record->image;
        } else if ($record) {
            $record->image_url = null;
        }

        return $record;
    }

    public function insert($data) {
        $this->db->insert($this->table, $data);
        return $this->db->insert_id();
    }

    public function update($id, $data) {
        $this->db->where('id', $id);
        return $this->db->update($this->table, $data);
    }

    public function delete($id) {
        $this->db->where('id', $id);
        return $this->db->delete($this->table);
    }

    public function exists($id) {
        $query = $this->db->get_where($this->table, array('id' => $id));
        return $query->num_rows() > 0;
    }
}
```

7. Tambahkan folder baru bernama views pada application\modules\Crudjs kemudian buat file baru bernama header.php. Isi program header.php:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><?php echo isset($title) ? $title : 'CRUD with AJAX'; ?> - CI3 CRUD</title
>   <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f5f5f5;
            line-height: 1.6;
        }

        .container {
            max-width: 1200px;
            margin: 0 auto;
            padding: 20px;
        }

        header {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            padding: 30px 0;
            margin-bottom: 30px;
            box-shadow: 0 4px 6px rgba(0,0,0,0.1);
        }

        header h1 {
            text-align: center;
            font-size: 2.5em;
            margin-bottom: 10px;
        }

        header p {
            text-align: center;
            font-size: 1.1em;
            opacity: 0.9;
        }

        .nav {
            background: white;
            padding: 15px 0;
            margin-bottom: 30px;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
            border-radius: 8px;
        }

        .nav ul {
            list-style: none;
            display: flex;
            justify-content: center;
            gap: 20px;
        }

        .nav a {
            color: #667eea;
            text-decoration: none;
            padding: 10px 20px;
            border-radius: 5px;
            transition: all 0.3s;
            font-weight: 500;
        }

        .nav a:hover {
            background: #667eea;
            color: white;
        }

        .alert {
            padding: 15px 20px;
            border-radius: 8px;
            margin-bottom: 20px;
            animation: slideDown 0.3s ease;
            display: none;
        }

        .alert.show {
            display: block;
        }

        @keyframes slideDown {
            from {
                opacity: 0;
                transform: translateY(-10px);
            }
            to {
                opacity: 1;
                transform: translateY(0);
            }
        }

        .alert-success {
            background-color: #d4edda;
            border-left: 4px solid #28a745;
            color: #155724;
        }
```

```css
.alert-error {
    background-color: #f8d7da;
    border-left: 4px solid #dc3545;
    color: #721c24;
}

.btn {
    display: inline-block;
    padding: 10px 20px;
    text-decoration: none;
    border-radius: 5px;
    cursor: pointer;
    border: none;
    font-size: 14px;
    font-weight: 500;
    transition: all 0.3s;
}

.btn-primary {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 10
0%);color: white;
}

.btn-primary:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 8px rgba(102, 126, 234, 0.4);
}

.btn-success {
    background-color: #28a745;
    color: white;
}

.btn-success:hover {
    background-color: #218838;
    transform: translateY(-2px);
    box-shadow: 0 4px 8px rgba(40, 167, 69, 0.4);
}

.btn-danger {
    background-color: #dc3545;
    color: white;
    border: none;
}

.btn-danger:hover {
    background-color: #c82333;
}

.btn-warning {
    background-color: #ffc107;
    color: #212529;
}

.btn-warning:hover {
    background-color: #e0a800;
}

.btn-info {
    background-color: #17a2b8;
    color: white;
}

.btn-info:hover {
    background-color: #138496;
}

.btn-secondary {
    background-color: #6c757d;
    color: white;
}

.btn-secondary:hover {
    background-color: #5a6268;
}

.card {
    background: white;
    border-radius: 10px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    padding: 30px;
    margin-bottom: 20px;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    margin-bottom: 8px;
    font-weight: 600;
    color: #333;
}

.form-group input[type="text"],
.form-group textarea,
.form-group input[type="file"] {
    width: 100%;
    padding: 12px;
    border: 2px solid #e0e0e0;
    border-radius: 6px;
    font-size: 14px;
    transition: border-color 0.3s;
    font-family: inherit;
}
```

```css
.form-group input[type="text"]:focus,
.form-group textarea:focus {
    outline: none;
    border-color: #667eea;
}

.form-group textarea {
    min-height: 150px;
    resize: vertical;
}

table {
    width: 100%;
    border-collapse: collapse;
    background: white;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

table thead {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 10
0%);color: white;
}

table th {
    padding: 15px;
    text-align: left;
    font-weight: 600;
}

table td {
    padding: 15px;
    border-bottom: 1px solid #e0e0e0;
}

table tbody tr:hover {
    background-color: #f8f9fa;
}

table tbody tr:last-child td {
    border-bottom: none;
}

.actions {
    display: flex;
    gap: 5px;
}

.post-image {
    max-width: 100px;
    height: auto;
    border-radius: 5px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.post-detail-image {
    max-width: 500px;
    width: 100%;
    height: auto;
    border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);
    margin: 20px 0;
}

.text-center {
    text-align: center;
}

.article-preview {
    max-width: 200px;
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
}

footer {
    background: #333;
    color: white;
    text-align: center;
    padding: 20px 0;
    margin-top: 50px;
}

.back-link {
    display: inline-block;
    margin-bottom: 20px;
}

.modal {
    display: none;
    position: fixed;
    z-index: 1000;
    left: 0;
    top: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    animation: fadeIn 0.3s ease;
}
```

```
        @keyframes fadeIn {
            from { opacity: 0; }
            to { opacity: 1; }
        }

        .modal.show {
            display: flex;
            align-items: center;
            justify-content: center;
        }

        .modal-content {
            background-color: white;
            padding: 30px;
            border-radius: 10px;
            width: 90%;
            max-width: 600px;
            box-shadow: 0 4px 20px rgba(0,0,0,0.3);
            animation: slideUp 0.3s ease;
        }

        @keyframes slideUp {
            from {
                opacity: 0;
                transform: translateY(50px);
            }
            to {
                opacity: 1;
                transform: translateY(0);
            }
        }

        .modal-header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            margin-bottom: 20px;
            padding-bottom: 15px;
            border-bottom: 2px solid #e0e0e0;
        }

        .modal-header h2 {
            margin: 0;
            color: #333;
        }

        .close {
            font-size: 28px;
            font-weight: bold;
            color: #999;
            cursor: pointer;
            transition: color 0.3s;
        }

        .close:hover {
            color: #333;
        }

        .modal-body {
            margin-bottom: 20px;
        }

        .modal-footer {
            display: flex;
            gap: 10px;
            justify-content: flex-end;
        }

        .loading {
            display: none;
            text-align: center;
            padding: 20px;
            color: #667eea;
        }

        .loading.show {
            display: block;
        }

        .spinner {
            border: 4px solid #f3f3f3;
            border-top: 4px solid #667eea;
            border-radius: 50%;
            width: 40px;
            height: 40px;
            animation: spin 1s linear infinite;
            margin: 0 auto 10px;
        }

        @keyframes spin {
            0% { transform: rotate(0deg); }
            100% { transform: rotate(360deg); }
        }

        .header-actions {
            display: flex;
            justify-content: space-between;
            align-items: center;
            margin-bottom: 20px;
        }

        .header-actions h2 {
            margin: 0;
        }

        small {
            display: block;
            margin-top: 5px;
            color: #666;
        }

    </style>
</head>
<body>
    <header>
        <div class="contalner">
            <h1>📄 CRUD with AJAX</h1>
            <p>CodeIgniter 3 CRUD Application - No Page Refresh</p>
        </div>
    </header>

    <div class="container">
        <div class="nav">
            <ul>
                <li><a href="<?php echo base_url(); ?>">Home</a></li>
                <li><a href="<?php echo base_url('crudjs'); ?>">CRUD with AJAX</a></li
>           </ul>
        </div>

        <div id="alertContainer">
            <div id="successAlert" class="alert alert-success"></div>
            <div id="errorAlert" class="alert alert-error"></div>
        </div>
```

8. Pada folder views application\modules\Crudjs\views buat file baru bernama index.php. Isi program index.php:

```php
<?php $this->load->view('crudjs/header'); ?>

<div class="card">
    <div class="header-actions">
        <h2>All Records</h2>
        <button type="button" class="btn btn-success" onclick="openCreateModal()">+ Create New Record</button
>   </div>

    <table>
        <thead>
            <tr>
                <th style="width: 50px;">ID</th>
                <th style="width: 100px;">Image</th>
                <th>Title</th>
                <th>Author</th>
                <th style="width: 250px;">Article Preview</th>
                <th style="width: 200px;">Actions</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td colspan="6" style="text-align: center; padding: 40px;">Loading...</td>
            </tr>
        </tbody>
    </table>
</div>

<?php $this->load->view('crudjs/footer'); ?>
```

9. Pada folder views application\modules\Crudjs\views buat file baru bernama footer.php. Isi program footer.php:

```
</div> <!-- End container -->

<footer>
    <div class="container">
        <p>&copy; <?php echo date('Y'); ?> CRUD with AJAX - Built with CodeIgniter 3</p>
    </div>
</footer>

<div id="formModal" class="modal">
    <div class="modal-content">
        <div class="modal-header">
            <h2 id="modalTitle">Create New Record</h2>
            <span class="close" onclick="closeModal()">&times;</span>
        </div>
        <div class="modal-body">
            <div class="loading" id="loading">
                <div class="spinner"></div>
                <p>Processing...</p>
            </div>
            <form id="crudForm" enctype="multipart/form-data">
                <input type="hidden" id="recordId" name="recordId">

                <div class="form-group">
                    <label for="title">Title <span style="color: red;">*</span></label>
                    <input type="text"
                           name="title"
                           id="title"
                           placeholder="Enter title"
                           required>
                </div>

                <div class="form-group">
                    <label for="author">Author <span style="color: red;">*</span></label>
                    <input type="text"
                           name="author"
                           id="author"
                           placeholder="Enter author name"
                           required>
                </div>

                <div class="form-group">
                    <label for="article">Article <span style="color: red;">*</span></label>
                    <textarea name="article"
                              id="article"
                              placeholder="Write your content here..."
                              required></textarea>
                </div>

                <div class="form-group">
                    <label for="image">Image</label>
                    <div id="currentImageContainer" style="display: none; margin-bottom: 10px;">
                        <p style="margin-bottom: 10px; font-weight: 500;">Current Image:</p>
                        <img id="currentImage"
                             alt="Current image"
                             style="max-width: 200px; border-radius: 8px; box-shadow: 0 2px 4px rgba(0,0,0,0.1);">
                    </div>
                    <input type="file"
                           name="image"
                           id="image"
                           accept="image/jpeg,image/jpg,image/png">
                    <small>Allowed formats: JPG, JPEG, PNG. Maximum size: 2MB</small>
                </div>
            </form>
        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-secondary" onclick="closeModal()">Cancel</button>
            <button type="button" class="btn btn-success" onclick="submitForm()">Save</button>
        </div>
    </div>
</div>

<div id="detailModal" class="modal">
    <div class="modal-content">
        <div class="modal-header">
            <h2 id="detailTitle">Record Details</h2>
            <span class="close" onclick="closeDetailModal()">&times;</span>
        </div>
        <div class="modal-body">
            <div class="loading" id="detailLoading">
                <div class="spinner"></div>
                <p>Loading...</p>
            </div>
            <div id="detailContent" style="display: none;">
                <div id="detailData"></div>
            </div>
        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-warning" onclick="editFromDetail()">Edit</button>
            <button type="button" class="btn btn-danger" onclick="deleteFromDetail()">Delete</button>
            <button type="button" class="btn btn-secondary" onclick="closeDetailModal()">Close</button>
        </div>
    </div>
</div>

<script>
    const baseUrl = '<?php echo base_url(); ?>';
    let isEditing = false;
    let currentRecordId = null;

    document.addEventListener('DOMContentLoaded', function() {
        console.log('CRUDjs Initialized - Base URL:', baseUrl);
        loadRecords();
    }, false);

    function loadRecords() {
        const url = baseUrl + 'crudjs/get_all';
        console.log('Loading records from:', url);

        fetch(url, {
            method: 'GET',
            headers: {
                'Content-Type': 'application/json'
            }
        })
        .then(response => {
            if (!response.ok) {
                throw new Error(`HTTP error! status: ${response.status}`);
            }
            return response.json();
        })
        .then(data => {
            console.log('Records loaded:', data);
            if (data.status === 'success') {
                renderTable(data.data);
            } else if (data.status === 'error') {
                showError(data.message || 'Failed to load records');
            }
        })
        .catch(error => {
            showError('Failed to load records: ' + error.message);
            console.error('Load records error:', error);
        });
    }

    function renderTable(records) {
        const tbody = document.querySelector('table tbody');
        if (!tbody) return;

        tbody.innerHTML = '';

        if (records.length === 0) {
            tbody.innerHTML = '<tr><td colspan="6" style="text-align: center; padding: 40px;">No records found. Create your first record!</td></tr>';
            return;
        }
```

```
    records.forEach(record => {
        const row = document.createElement('tr');
        const imageHtml = record.image_url ? `<img src="${record.image_url}" alt="${record.title}" class="post-image">` : '<span style="color: #999;">No image</span
>';    const articlePreview = record.article.substring(0, 100) + (record.article.length > 100 ? '...' : '');

        row.innerHTML = `
            <td>${record.id}</td>
            <td>${imageHtml}</td>
            <td><strong>${escapeHtml(record.title)}</strong></td>
            <td>${escapeHtml(record.author)}</td>
            <td>
                <div class="article-preview">${escapeHtml(articlePreview)}</div>
            </td>
            <td>
                <div class="actions">
                    <button type="button" class="btn btn-info" onclick="viewRecord(${record.id})">View</button>
                    <button type="button" class="btn btn-warning" onclick="editRecord(${record.id})">Edit</button>
                    <button type="button" class="btn btn-danger" onclick="deleteRecord(${record.id})">Delete</button>
                </div>
            </td>
        `;
        tbody.appendChild(row);
    });
}

function openCreateModal() {
    isEditing = false;
    currentRecordId = null;
    document.getElementById('modalTitle').textContent = 'Create New Record';
    document.getElementById('crudForm').reset();
    document.getElementById('recordId').value = '';
    document.getElementById('currentImageContainer').style.display = 'none';
    document.getElementById('formModal').classList.add('show');
}

function editRecord(id) {
    isEditing = true;
    currentRecordId = id;
    document.getElementById('modalTitle').textContent = 'Edit Record';
    document.getElementById('recordId').value = id;

    const url = baseUrl + 'crudjs/get_record/' + id;
    console.log('Loading record from:', url);

    fetch(url, {
        method: 'GET',
        headers: {
            'Content-Type': 'application/json'
        }
    })
    .then(response => {
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
        return response.json();
    })
    .then(data => {
        console.log('Record loaded:', data);
        if (data.status === 'success') {
            const record = data.data;
            document.getElementById('title').value = record.title;
            document.getElementById('author').value = record.author;
            document.getElementById('article').value = record.article;

            if (record.image_url) {
                document.getElementById('currentImage').src = record.image_url;
                document.getElementById('currentImageContainer').style.display = 'block';
            } else {
                document.getElementById('currentImageContainer').style.display = 'none';
            }

            document.getElementById('formModal').classList.add('show');
        } else {
            showError(data.message || 'Failed to load record');
        }
    })
    .catch(error => {
        showError('Failed to load record: ' + error.message);
        console.error('Load record error:', error);
    });
}

function submitForm() {
    const form = document.getElementById('crudForm');
    const formData = new FormData(form);

    const recordId = document.getElementById('recordId').value;
    const url = isEditing ? baseUrl + 'crudjs/update/' + recordId : baseUrl + 'crudjs/store';

    console.log('Submitting form to:', url, 'Edit mode:', isEditing);
    document.getElementById('loading').classList.add('show');

    fetch(url, {
        method: 'POST',
        body: formData
    })
    .then(response => {
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
        return response.json();
    })
    .then(data => {
        console.log('Form submitted, response:', data);
        document.getElementById('loading').classList.remove('show');

        if (data.status === 'success') {
            showSuccess(data.message);
            closeModal();
            loadRecords();
        } else {
            showError(data.message || 'Operation failed');
        }
    })
    .catch(error => {
        console.error('Submit error:', error);
        document.getElementById('loading').classList.remove('show');
        showError('An error occurred: ' + error.message);
    });
}

function viewRecord(id) {
    console.log('Viewing record:', id);
    document.getElementById('detailModal').classList.add('show');
    document.getElementById('detailLoading').classList.add('show');
    document.getElementById('detailContent').style.display = 'none';

    const url = baseUrl + 'crudjs/get_record/' + id;
    console.log('Loading detail from:', url);

    fetch(url, {
        method: 'GET',
        headers: {
            'Content-Type': 'application/json'
        }
    })
    .then(response => {
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
        return response.json();
    })
    .then(data => {
        console.log('Detail loaded:', data);
        if (data.status === 'success') {
            const record = data.data;
            currentRecordId = record.id;
            renderDetailView(record);
        } else {
            showError(data.message || 'Failed to load record details');
            document.getElementById('detailLoading').classList.remove('show');
        }
    })
    .catch(error => {
        console.error('Load detail error:', error);
        showError('Failed to load record: ' + error.message);
        document.getElementById('detailLoading').classList.remove('show');
    });
}
```

```
        function renderDetailView(record) {
            const detailData = document.getElementById('detailData');
            let detailHtml = `
                <h3>${escapeHtml(record.title)}</h3>
                <hr style="margin: 20px 0; border: none; border-top: 2px solid #e0e0e0;">
                <p><strong>Author:</strong> ${escapeHtml(record.author)}</p>
                <p><strong>Created:</strong> ${new Date(record.created_at).toLocaleString()}</p>
            `;

            if (record.updated_at && record.updated_at !== record.created_at) {
                detailHtml += `<p><strong>Updated:</strong> ${new Date(record.updated_at).toLocaleString()}</p>
`;            }

            if (record.image_url) {
                detailHtml += `<img src="${record.image_url}" alt="${record.title}" class="post-detail-image">`;
            }

            detailHtml += `
                <div style="margin-top: 30px;">
                    <h4 style="margin-bottom: 15px; color: #333;">Content</h4>
                    <div style="line-height: 1.8; color: #555; white-space: pre-wrap;">
                        ${escapeHtml(record.article)}
                    </div>
                </div>
            `;

            detailData.innerHTML = detailHtml;
            document.getElementById('detailLoading').classList.remove('show');
            document.getElementById('detailContent').style.display = 'block';
            document.getElementById('detailTitle').textContent = 'Record Details';
        }

        function deleteRecord(id) {
            if (!confirm('Are you sure you want to delete this record?')) {
                return;
            }

            console.log('Deleting record:', id);
            const url = baseUrl + 'crudjs/delete/' + id;
            console.log('Delete URL:', url);

            fetch(url, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                }
            })
            .then(response => {
                if (!response.ok) {
                    throw new Error(`HTTP error! status: ${response.status}`);
                }
                return response.json();
            })
            .then(data => {
                console.log('Delete response:', data);
                if (data.status === 'success') {
                    showSuccess(data.message);
                    loadRecords();
                } else {
                    showError(data.message || 'Failed to delete record');
                }
            })
            .catch(error => {
                console.error('Delete error:', error);
                showError('Failed to delete record: ' + error.message);
            });
        }

        function deleteFromDetail() {
            if (!currentRecordId) return;

            if (!confirm('Are you sure you want to delete this record?')) {
                return;
            }

            deleteRecord(currentRecordId);
            closeDetailModal();
        }

        function editFromDetail() {
            if (!currentRecordId) return;

            closeDetailModal();
            editRecord(currentRecordId);
        }

        function closeModal() {
            document.getElementById('formModal').classList.remove('show');
        }

        function closeDetailModal() {
            document.getElementById('detailModal').classList.remove('show');
        }

        function showSuccess(message) {
            console.log('Success:', message);
            const alert = document.getElementById('successAlert');
            alert.textContent = '✓ ' + message;
            alert.classList.add('show');
            setTimeout(() => {
                alert.classList.remove('show');
            }, 4000);
        }

        function showError(message) {
            console.error('Error:', message);
            const alert = document.getElementById('errorAlert');
            alert.textContent = '✗ ' + message;
            alert.classList.add('show');
            setTimeout(() => {
                alert.classList.remove('show');
            }, 6000);
        }

        function escapeHtml(text) {
            const div = document.createElement('div');
            div.textContent = text;
            return div.innerHTML;
        }

        window.onclick = function(event) {
            const formModal = document.getElementById('formModal');
            const detailModal = document.getElementById('detailModal');

            if (event.target === formModal) {
                closeModal();
            }
            if (event.target === detailModal) {
                closeDetailModal();
            }
        }
    </script>
</body>
</html>
```

10. Buat file Jwt.php pada folder application\libraries. Isi program Jwt.php:

```php
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Jwt {
    private $secret_key = 'your_secret_key_change_this_in_production';
    private $algorithm = 'HS256';
    private $expiration = 604800; // 7 days in seconds

    public function __construct()
    {
        $this->secret_key = config_item('jwt_secret_key') ?: $this->secret_key;
        $this->expiration = config_item('jwt_expiration') ?: $this->expiration;
    }

    /**
     * Create JWT token
     * @param array $payload Data to encode in token
     * @return string JWT token
     */
    public function create($payload = array())
    {
        $header = array(
            'typ' => 'JWT',
            'alg' => $this->algorithm
        );

        $payload['iat'] = time();
        $payload['exp'] = time() + $this->expiration;

        $header_encoded = $this->base64_url_encode(json_encode($header));
        $payload_encoded = $this->base64_url_encode(json_encode($payload));

        $signature = hash_hmac(
            'sha256',
            $header_encoded . '.' . $payload_encoded,
            $this->secret_key,
            true
        );
        $signature_encoded = $this->base64_url_encode($signature);

        return $header_encoded . '.' . $payload_encoded . '.' . $signature_encoded;
    }

    /**
     * Verify and decode JWT token
     * @param string $token JWT token to verify
     * @return object|false Decoded payload or false if invalid
     */
    public function verify($token = '')
    {
        if (empty($token)) {
            return false;
        }

        $parts = explode('.', $token);
        if (count($parts) !== 3) {
            return false;
        }

        list($header_encoded, $payload_encoded, $signature_encoded) = $parts;

        $signature = hash_hmac(
            'sha256',
            $header_encoded . '.' . $payload_encoded,
            $this->secret_key,
            true
        );
        $signature_computed = $this->base64_url_encode($signature);

        if ($signature_computed !== $signature_encoded) {
            return false;
        }

        $payload = json_decode($this->base64_url_decode($payload_encoded));

        if (isset($payload->exp) && $payload->exp < time()) {
            return false;
        }

        return $payload;
    }

    /**
     * Get token from request header
     * Authorization: Bearer <token>
     * @return string|false Token or false if not found
     */
    public function get_token_from_request()
    {

        $auth_header = null;

        if (function_exists('apache_request_headers')) {
            $headers = apache_request_headers();
            $auth_header = isset($headers['Authorization']) ? $headers['Authorization'] : nul
1;      }

        if (!$auth_header && isset($_SERVER['HTTP_AUTHORIZATION'])) {
            $auth_header = $_SERVER['HTTP_AUTHORIZATION'];
        }

        if (!$auth_header && isset($_SERVER['REDIRECT_HTTP_AUTHORIZATION'])) {
            $auth_header = $_SERVER['REDIRECT_HTTP_AUTHORIZATION'];
        }

        if (!$auth_header) {
            return false;
        }

        if (preg_match('/Bearer\s+(\S+)/', $auth_header, $matches)) {
            return $matches[1];
        }

        return false;
    }

    private function base64_url_encode($input)
    {
        return str_replace(
            array('+', '/', '='),
            array('-', '_', ''),
            base64_encode($input)
        );
    }

    private function base64_url_decode($input)
    {
        $remainder = strlen($input) % 4;
        if ($remainder) {
            $input .= str_repeat('=', 4 - $remainder);
        }

        return base64_decode(
            str_replace(
                array('-', '_'),
                array('+', '/'),
                $input
            )
        );
    }
}
```

11. Buat folder Api pada application\modules, kemudian buat folder controllers di dalamnya lalu buat file bernama Auth.php. Isi program Auth.php:

```php
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Auth extends MX_Controller {

    public function __construct()
    {
        parent::__construct();
        $this->load->model('User_model');
        $this->load->library('jwt');
        $this->load->library('form_validation');
    }

    public function register()
    {
        header('Content-Type: application/json; charset=utf-8');
        ob_clean();

        try {
            $input = json_decode(file_get_contents('php://input'), true);

            if (empty($input['name']) || empty($input['email']) || empty($input['password']))
{
                http_response_code(400);
                echo json_encode([
                    'status' => false,
                    'message' => 'Name, email, and password are required'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            if (!filter_var($input['email'], FILTER_VALIDATE_EMAIL)) {
                http_response_code(400);
                echo json_encode([
                    'status' => false,
                    'message' => 'Invalid email format'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            if ($this->User_model->email_exists($input['email'])) {
                http_response_code(409);
                echo json_encode([
                    'status' => false,
                    'message' => 'Email already registered'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            if (strlen($input['password']) < 6) {
                http_response_code(400);
                echo json_encode([
                    'status' => false,
                    'message' => 'Password must be at least 6 characters'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $user_data = [
                'name' => htmlspecialchars($input['name']),
                'email' => strtolower($input['email']),
                'password' => password_hash($input['password'], PASSWORD_BCRYPT)
            ];

            $user_id = $this->User_model->create($user_data);

            if (!$user_id) {
                http_response_code(500);
                echo json_encode([
                    'status' => false,
                    'message' => 'Failed to create user'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $user = $this->User_model->get_by_id($user_id);

            $token = $this->jwt->create([
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email
            ]);

            http_response_code(201);
            echo json_encode([
                'status' => true,
                'message' => 'User registered successfully',
                'data' => [
                    'id' => $user->id,
                    'name' => $user->name,
                    'email' => $user->email,
                    'created_at' => $user->created_at
                ],
                'access_token' => $token,
                'token_type' => 'Bearer'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;

        } catch (Exception $e) {
            http_response_code(500);
            echo json_encode([
                'status' => false,
                'message' => 'Server error: ' . $e->getMessage()
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }
    }

    public function login()
    {
        header('Content-Type: application/json; charset=utf-8');
        ob_clean();

        try {
            $input = json_decode(file_get_contents('php://input'), true);

            if (empty($input['email']) || empty($input['password'])) {
                http_response_code(400);
                echo json_encode([
                    'status' => false,
                    'message' => 'Email and password are required'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $user = $this->User_model->get_by_email(strtolower($input['email']));

            if (!$user) {
                http_response_code(401);
                echo json_encode([
                    'status' => false,
                    'message' => 'Invalid credentials'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            if (!password_verify($input['password'], $user->password)) {
                http_response_code(401);
                echo json_encode([
                    'status' => false,
                    'message' => 'Invalid credentials'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $token = $this->jwt->create([
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email
            ]);
```

```php
            http_response_code(200);
            echo json_encode([
                'status' => true,
                'message' => 'Login successful',
                'data' => [
                    'id' => $user->id,
                    'name' => $user->name,
                    'email' => $user->email
                ],
                'access_token' => $token,
                'token_type' => 'Bearer'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;

        } catch (Exception $e) {
            http_response_code(500);
            echo json_encode([
                'status' => false,
                'message' => 'Server error: ' . $e->getMessage()
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }
    }

    public function logout()
    {
        header('Content-Type: application/json; charset=utf-8');
        ob_clean();

        try {
            $token = $this->jwt->get_token_from_request();

            if (!$token) {
                http_response_code(401);
                echo json_encode([
                    'status' => false,
                    'message' => 'No token provided'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $decoded = $this->jwt->verify($token);

            if (!$decoded) {
                http_response_code(401);
                echo json_encode([
                    'status' => false,
                    'message' => 'Invalid token'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $user = $this->User_model->get_by_id($decoded->id);

            if (!$user) {
                http_response_code(404);
                echo json_encode([
                    'status' => false,
                    'message' => 'User not found'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            http_response_code(200);
            echo json_encode([
                'status' => true,
                'message' => 'Logout successful'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;

        } catch (Exception $e) {
            http_response_code(500);
            echo json_encode([
                'status' => false,
                'message' => 'Server error: ' . $e->getMessage()
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }
    }

    public function me()
    {
        header('Content-Type: application/json; charset=utf-8');
        ob_clean();

        try {
            $token = $this->jwt->get_token_from_request();

            if (!$token) {
                http_response_code(401);
                echo json_encode([
                    'status' => false,
                    'message' => 'No token provided'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $decoded = $this->jwt->verify($token);

            if (!$decoded) {
                http_response_code(401);
                echo json_encode([
                    'status' => false,
                    'message' => 'Invalid or expired token'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $user = $this->User_model->get_by_id($decoded->id);

            if (!$user) {
                http_response_code(404);
                echo json_encode([
                    'status' => false,
                    'message' => 'User not found'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            http_response_code(200);
            echo json_encode([
                'status' => true,
                'message' => 'User retrieved successfully',
                'data' => [
                    'id' => $user->id,
                    'name' => $user->name,
                    'email' => $user->email,
                    'created_at' => $user->created_at,
                    'updated_at' => $user->updated_at
                ]
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;

        } catch (Exception $e) {
            http_response_code(500);
            echo json_encode([
                'status' => false,
                'message' => 'Server error: ' . $e->getMessage()
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }
    }
}
```

12. Pada folder application\modules\Api\controllers, buat file bernama Post.php. Isi program Post.php:

```php
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Post extends MX_Controller {

    public function __construct()
    {
        parent::__construct();
        $this->load->model('Post_model');
        $this->load->library('jwt');
        $this->load->library('upload');
    }

    private function parse_multipart_form()
    {
        $content_type = $_SERVER['CONTENT_TYPE'] ?? '';
        $input = [];

        if (strpos($content_type, 'multipart/form-data') !== false) {
            $boundary = '';
            if (preg_match('/boundary=([^;]+)/', $content_type, $matches)) {
                $boundary = trim($matches[1], '"');
            }

            if (!$boundary) {
                return [];
            }

            $raw = file_get_contents('php://input');
            $parts = preg_split('/' . preg_quote("--$boundary") . '/', $raw);

            foreach ($parts as $part) {
                if (empty(trim($part)) || $part === "--\r\n" || $part === "--") {
                    continue;
                }

                $split = preg_split("/\r\n\r\n/", trim($part), 2);
                if (count($split) !== 2) {
                    continue;
                }

                $headers = $split[0];
                $content = rtrim($split[1], "\r\n");

                if (preg_match('/name="([^"]+)"/', $headers, $matches)) {
                    $name = $matches[1];

                    if (preg_match('/filename="([^"]+)"/', $headers, $filename_matches)) {
                        $file_content_type = 'application/octet-stream';
                        if (preg_match('/Content-Type:\s*([^\r\n]+)/', $headers, $type_matches))
{
                            $file_content_type = trim($type_matches[1]);
                        }

                        $input[$name] = [
                            'name' => $filename_matches[1],
                            'type' => $file_content_type,
                            'content' => $content
                        ];
                    } else {
                        $input[$name] = $content;
                    }
                }
            }

            return $input;
        }

        if (strpos($content_type, 'application/x-www-form-urlencoded') !== false) {
            parse_str(file_get_contents('php://input'), $input);
            return $input ?? [];
        }

        return array_merge($_POST ?? [], $_FILES ?? []);
    }

    private function get_image_url($image_filename)
    {
        if (empty($image_filename)) {
            return null;
        }
        return 'http://localhost/ci3_project/uploads/posts/' . $image_filename;
    }

    public function handle($id = null)
    {
        $method = $_SERVER['REQUEST_METHOD'];

        if ($method === 'GET') {
            if ($id) {
                $this->show($id);
            } else {
                $this->index();
            }
        } elseif ($method === 'POST') {
            $this->store();
        } elseif ($method === 'PUT') {
            $this->update($id);
        } elseif ($method === 'DELETE') {
            $this->delete($id);
        } else {
            http_response_code(405);
            header('Content-Type: application/json; charset=utf-8');
            echo json_encode([
                'status' => false,
                'message' => 'Method not allowed'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }
    }

    public function index()
    {
        header('Content-Type: application/json; charset=utf-8');
        ob_clean();

        try {
            $page = isset($_GET['page']) ? max(1, (int)$_GET['page']) : 1;
            $per_page = isset($_GET['per_page']) ? max(1, min(100, (int)$_GET['per_page'])) : 10;
            $offset = ($page - 1) * $per_page;

            $posts = $this->Post_model->get_all($per_page, $offset);
            $total = $this->Post_model->count_all();

            foreach ($posts as &$post) {
                if (!empty($post->image)) {
                    $post->image_url = $this->get_image_url($post->image);
                } else {
                    $post->image_url = null;
                }
            }

            http_response_code(200);
            echo json_encode([
                'status' => true,
                'message' => 'Posts retrieved successfully',
                'data' => $posts,
                'pagination' => [
                    'total' => (int)$total,
                    'per_page' => $per_page,
                    'current_page' => $page,
                    'last_page' => ceil($total / $per_page)
                ]
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;

        } catch (Exception $e) {
            http_response_code(500);
            echo json_encode([
                'status' => false,
                'message' => 'Server error: ' . $e->getMessage()
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }
    }
```

```php
public function show($id)
{
    header('Content-Type: application/json; charset=utf-8');
    ob_clean();

    try {
        if (!is_numeric($id)) {
            http_response_code(400);
            echo json_encode([
                'status' => false,
                'message' => 'Invalid post ID'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $post = $this->Post_model->get_by_id($id);

        if (!$post) {
            http_response_code(404);
            echo json_encode([
                'status' => false,
                'message' => 'Post not found'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        if (!empty($post->image)) {
            $post->image_url = $this->get_image_url($post->image);
        } else {
            $post->image_url = null;
        }

        http_response_code(200);
        echo json_encode([
            'status' => true,
            'message' => 'Post retrieved successfully',
            'data' => $post
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;

    } catch (Exception $e) {
        http_response_code(500);
        echo json_encode([
            'status' => false,
            'message' => 'Server error: ' . $e->getMessage()
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;
    }
}

public function store()
{
    header('Content-Type: application/json; charset=utf-8');
    ob_clean();

    try {
        $token = $this->jwt->get_token_from_request();

        if (!$token) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'Unauthorized: No token provided'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $decoded = $this->jwt->verify($token);

        if (!$decoded) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'Unauthorized: Invalid or expired token'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $input = json_decode(file_get_contents('php://input'), true);

        if (!$input && !empty($_POST)) {
            $input = $_POST;
        }

        if (empty($input['title']) || empty($input['article'])) {
            http_response_code(400);
            echo json_encode([
                'status' => false,
                'message' => 'Title and article are required'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $post_data = [
            'title' => htmlspecialchars($input['title']),
            'author' => htmlspecialchars($input['author'] ?? 'Anonymou
s'),
            'article' => htmlspecialchars($input['article'])
        ];

        $image_name = '';
        if (!empty($_FILES['image']['name'])) {
            $image_name = $this->_upload_image();
            if (!$image_name) {
                http_response_code(400);
                echo json_encode([
                    'status' => false,
                    'message' => 'Image upload failed'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }
            $post_data['image'] = $image_name;
        }

        $post_id = $this->Post_model->create($post_data);

        if (!$post_id) {
            http_response_code(500);
            echo json_encode([
                'status' => false,
                'message' => 'Failed to create post'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $post = $this->Post_model->get_by_id($post_id);

        if (!empty($post->image)) {
            $post->image_url = $this->get_image_url($post->image);
        } else {
            $post->image_url = null;
        }

        http_response_code(201);
        echo json_encode([
            'status' => true,
            'message' => 'Post created successfully',
            'data' => $post
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;

    } catch (Exception $e) {
        http_response_code(500);
        echo json_encode([
            'status' => false,
            'message' => 'Server error: ' . $e->getMessage()
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;
    }
}
```

```php
public function update($id)
{
    header('Content-Type: application/json; charset=utf-8');
    ob_clean();

    try {
        $token = $this->jwt->get_token_from_request();

        if (!$token) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'Unauthorized: No token provided'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $decoded = $this->jwt->verify($token);

        if (!$decoded) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'Unauthorized: Invalid or expired token'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        if (!is_numeric($id)) {
            http_response_code(400);
            echo json_encode([
                'status' => false,
                'message' => 'invalid post ID'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        if (!$this->Post_model->exists($id)) {
            http_response_code(404);
            echo json_encode([
                'status' => false,
                'message' => 'Post not found'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $input = json_decode(file_get_contents('php://input'), true);
        if (!$input) {
            $input = $this->parse_multipart_form();

            if (!empty($input['image']) && is_array($input['image']) && isset($input['image']['content'])) {
                $_FILES['image'] = [
                    'name' => $input['image']['name'] ?? 'image_' . time() . '.png',
                    'type' => $input['image']['type'] ?? 'image/png',
                    'tmp_name' => $this->_save_temp_file($input['image']['content']),
                    'error' => 0,
                    'size' => strlen($input['image']['content'])
                ];
            }
        }

        if (empty($input['title']) || empty($input['article'])) {
            http_response_code(400);
            echo json_encode([
                'status' => false,
                'message' => 'Title and article are required'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $post_data = [
            'title' => htmlspecialchars($input['title']),
            'author' => htmlspecialchars($input['author'] ?? 'Anonymous'),
            'article' => htmlspecialchars($input['article'])
        ];

        if (!empty($_FILES['image']['name']) || (isset($input['image']) && is_array($input['image']) && !empty($input['image']['name'])))
{           $old_post = $this->Post_model->get_by_id($id);

            if (!empty($old_post->image)) {
                $old_file = FCPATH . 'uploads/posts/' . $old_post->image;
                if (file_exists($old_file)) {
                    unlink($old_file);
                }
            }

            $image_name = $this->_upload_image();
            if ($image_name) {
                $post_data['image'] = $image_name;
            }
        }

        $result = $this->Post_model->update($id, $post_data);

        if (!$result) {
            http_response_code(500);
            echo json_encode([
                'status' => false,
                'message' => 'Failed to update post'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $post = $this->Post_model->get_by_id($id);

        if (!empty($post->image)) {
            $post->image_url = $this->get_image_url($post->image);
        } else {
            $post->image_url = null;
        }

        http_response_code(200);
        echo json_encode([
            'status' => true,
            'message' => 'Post updated successfully',
            'data' => $post
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;

    } catch (Exception $e) {
        http_response_code(500);
        echo json_encode([
            'status' => false,
            'message' => 'Server error: ' . $e->getMessage()
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;
    }
}

public function delete($id)
{
    header('Content-Type: application/json; charset=utf-8');
    ob_clean();

    try {
        $token = $this->jwt->get_token_from_request();

        if (!$token) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'Unauthorized: No token provided'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }
```

```php
            $decoded = $this->jwt->verify($token);

            if (!$decoded) {
                http_response_code(401);
                echo json_encode([
                    'status' => false,
                    'message' => 'Unauthorized: Invalid or expired token'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            if (!is_numeric($id)) {
                http_response_code(400);
                echo json_encode([
                    'status' => false,
                    'message' => 'Invalid post ID'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            $post = $this->Post_model->get_by_id($id);

            if (!$post) {
                http_response_code(404);
                echo json_encode([
                    'status' => false,
                    'message' => 'Post not found'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            if (!empty($post->image)) {
                $file = FCPATH . 'uploads/posts/' . $post->image;
                if (file_exists($file)) {
                    unlink($file);
                }
            }

            $result = $this->Post_model->delete($id);

            if (!$result) {
                http_response_code(500);
                echo json_encode([
                    'status' => false,
                    'message' => 'Failed to delete post'
                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
                exit;
            }

            http_response_code(200);
            echo json_encode([
                'status' => true,
                'message' => 'Post deleted successfully'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;

        } catch (Exception $e) {
            http_response_code(500);
            echo json_encode([
                'status' => false,
                'message' => 'Server error: ' . $e->getMessage()
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }
    }

    private function _save_temp_file($content)
    {
        $temp_path = sys_get_temp_dir() . '/ci3_upload_' . uniqid() . '.tm
p';     file_put_contents($temp_path, $content);
        return $temp_path;
    }

    private function _upload_image()
    {
        $config['upload_path'] = FCPATH . 'uploads/posts/';
        $config['allowed_types'] = 'gif|jpg|png|jpeg';
        $config['max_size'] = 2048; // 2MB
        $config['file_name'] = 'post_' . time() . '_' . uniqid();

        if (!is_dir($config['upload_path'])) {
            mkdir($config['upload_path'], 0755, true);
        }

        $this->upload->initialize($config);

        if (!$this->upload->do_upload('image')) {
            return false;
        }

        $upload_data = $this->upload->data();
        return $upload_data['file_name'];
    }
}
```

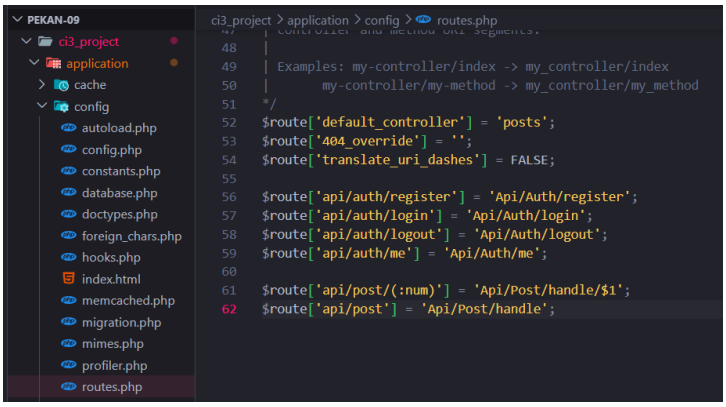
13. Pada folder application\modules\Api, buat folder baru bernama models lalu buat file bernama Post_model.php pada folder tersebut. Isi program Post_model.php:

```php
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Post_model extends CI_Model {

    private $table = 'posts';

    public function __construct()
    {
        parent::__construct();
        $this->load->database();
    }

    public function get_all($limit = 10, $offset = 0)
    {
        $query = $this->db
            ->select('id, title, author, article, image, created_at, updated_at')
            ->limit($limit, $offset)
            ->order_by('created_at', 'DESC')
            ->get($this->table);
        return $query->result();
    }

    public function count_all()
    {
        return $this->db->count_all($this->table);
    }

    public function get_by_id($id)
    {
        $query = $this->db
            ->select('id, title, author, article, image, created_at, updated_at')
            ->get_where($this->table, array('id' => $id));
        return $query->row();
    }

    public function create($data)
    {
        $data['created_at'] = date('Y-m-d H:i:s');
        $data['updated_at'] = date('Y-m-d H:i:s');

        $this->db->insert($this->table, $data);
        return $this->db->insert_id();
    }

    public function update($id, $data)
    {
        $data['updated_at'] = date('Y-m-d H:i:s');
        $this->db->where('id', $id);
        return $this->db->update($this->table, $data);
    }

    public function delete($id)
    {
        $this->db->where('id', $id);
        return $this->db->delete($this->table);
    }

    public function exists($id)
    {
        $query = $this->db->get_where($this->table, array('id' => $id));
        return $query->num_rows() > 0;
    }

    public function search($keyword, $limit = 10, $offset = 0)
    {
        $this->db->like('title', $keyword);
        $this->db->or_like('description', $keyword);
        $this->db->limit($limit, $offset);
        $this->db->order_by('created_at', 'DESC');
        $query = $this->db->get($this->table);
        return $query->result();
    }

    public function get_by_user($user_id, $limit = 10, $offset = 0)
    {
        $query = $this->db
            ->select('id, title, author, article, image, created_at, updated_at')
            ->where('user_id', $user_id)
            ->limit($limit, $offset)
            ->order_by('created_at', 'DESC')
            ->get($this->table);
        return $query->result();
    }
}
```

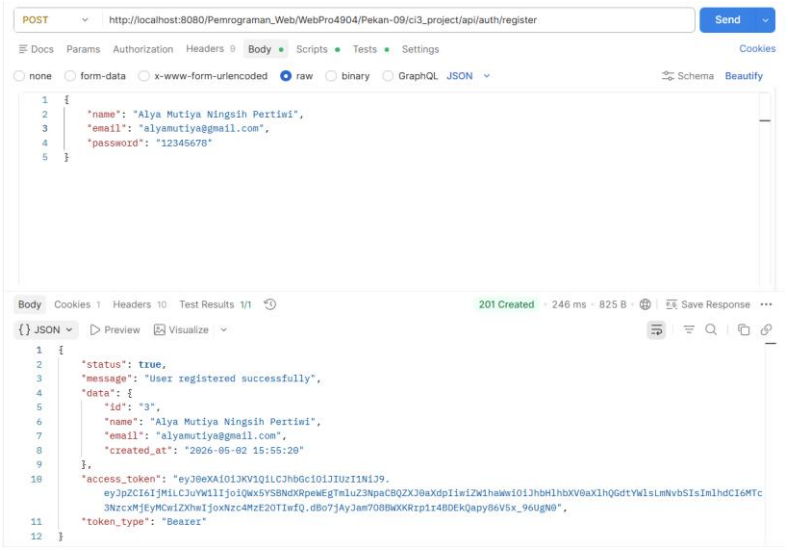
14. Pada folder application\modules\Api\models, buat file bernama User_model.php. Isi program User_model.php:

```php
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class User_model extends CI_Model {

    private $table = 'users';

    public function __construct()
    {
        parent::__construct();
        $this->load->database();
    }

    public function get_all()
    {
        $query = $this->db->get($this->table);
        return $query->result();
    }

    public function get_by_id($id)
    {
        $query = $this->db->get_where($this->table, array('id' => $id));
        return $query->row();
    }

    public function get_by_email($email)
    {
        $query = $this->db->get_where($this->table, array('email' => $emai
l));    return $query->row();
    }

    public function create($data)
    {
        $data['created_at'] = date('Y-m-d H:i:s');
        $data['updated_at'] = date('Y-m-d H:i:s');

        $this->db->insert($this->table, $data);
        return $this->db->insert_id();
    }

    public function update($id, $data)
    {
        $data['updated_at'] = date('Y-m-d H:i:s');
        $this->db->where('id', $id);
        return $this->db->update($this->table, $data);
    }

    public function delete($id)
    {
        $this->db->where('id', $id);
        return $this->db->delete($this->table);
    }

    public function email_exists($email)
    {
        $query = $this->db->get_where($this->table, array('email' => $emai
l));    return $query->num_rows() > 0;
    }

    public function exists($id)
    {
        $query = $this->db->get_where($this->table, array('id' => $id));
        return $query->num_rows() > 0;
    }
}
```

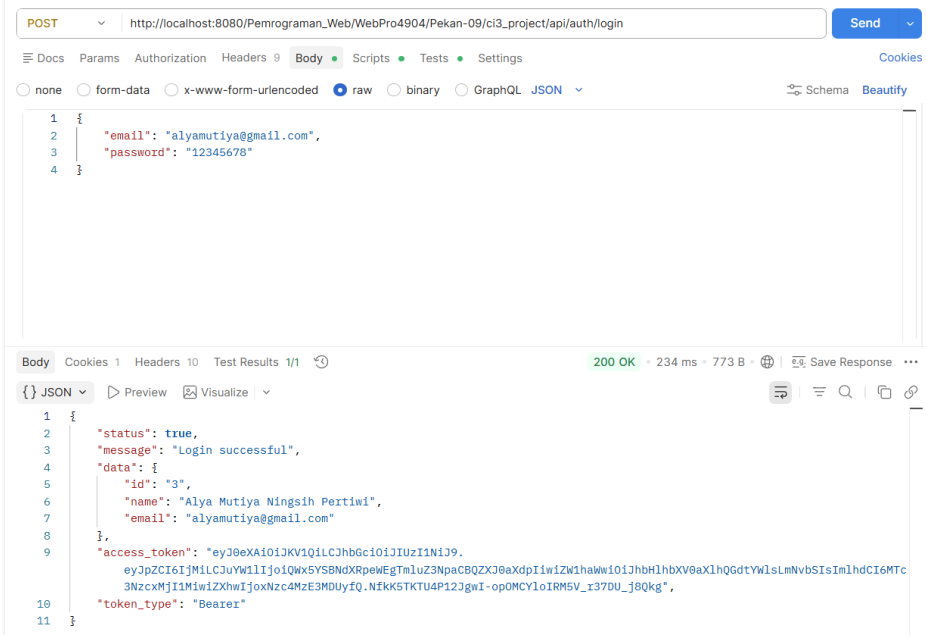
15. Buka application/config/routes.php kemudian tambahkan program berikut.



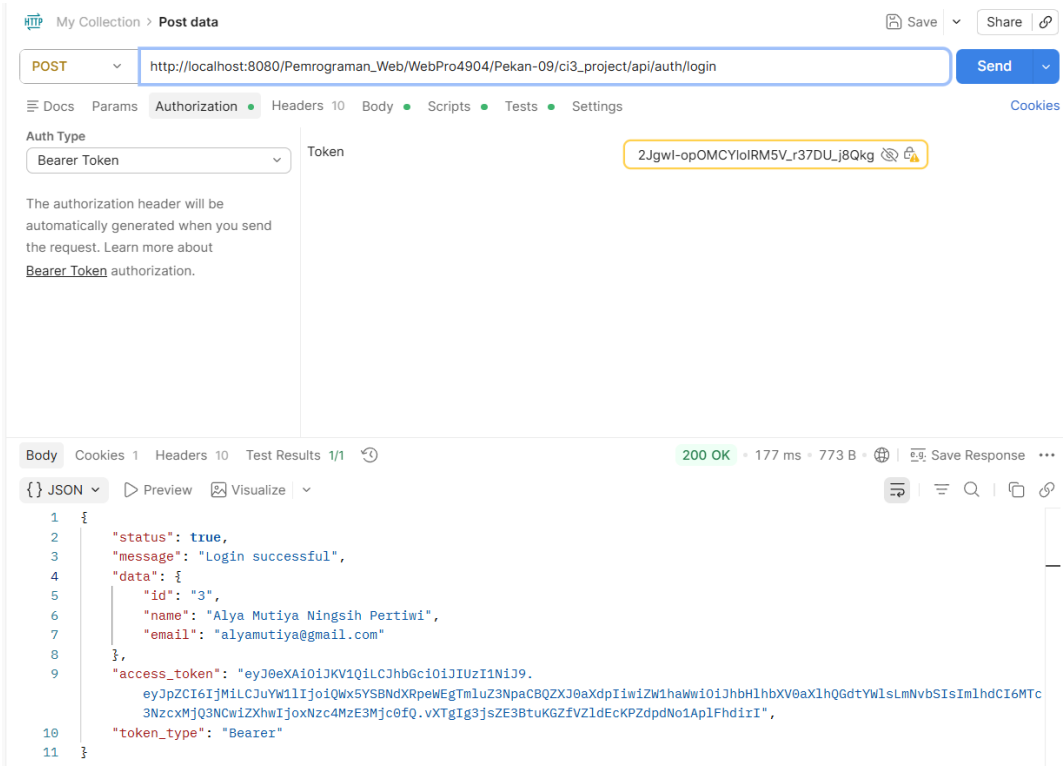
16. Buat folder uploads untuk menyediakan tempat penyimpanan file yang di upload oleh pengguna.
17. Jalankan Aplikasi Restfull API dengan Postman.
    a. Register



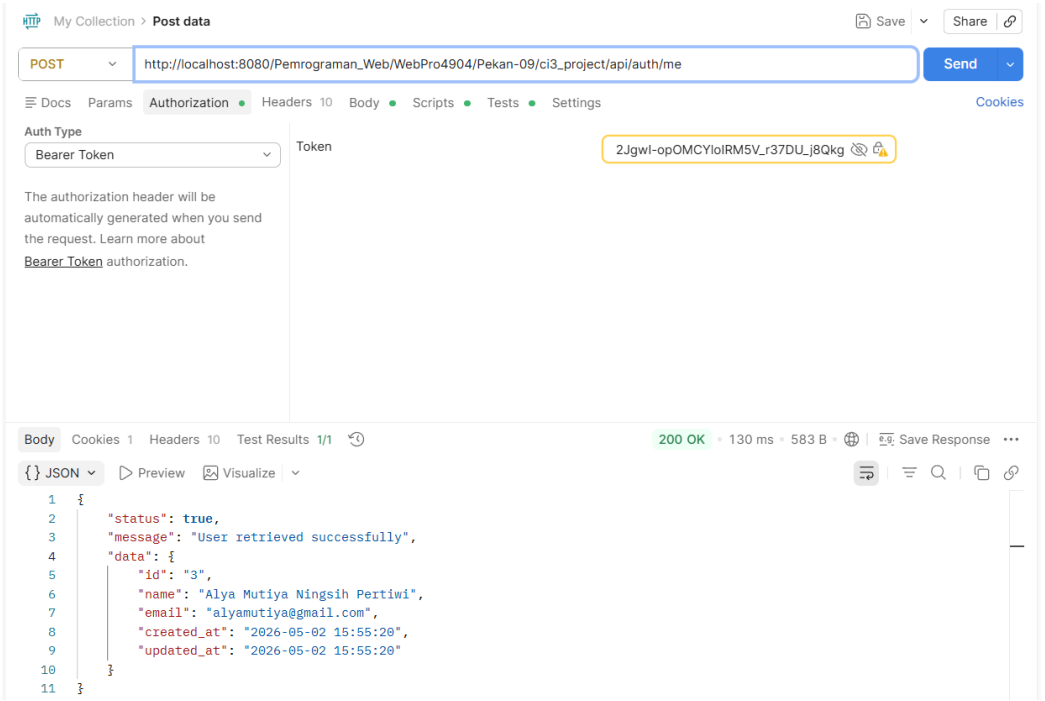
    b. Login



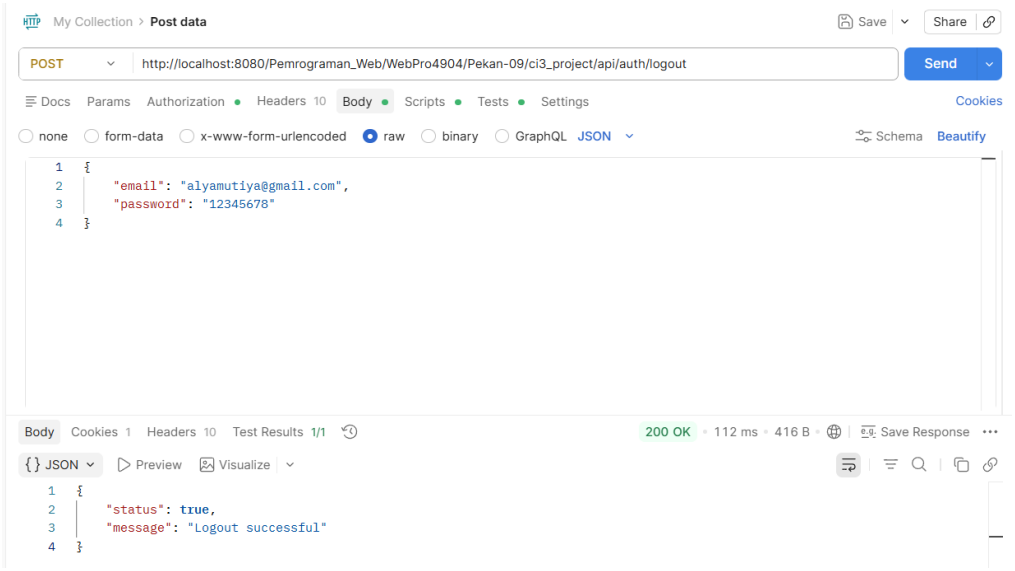
c. Token keamanan login



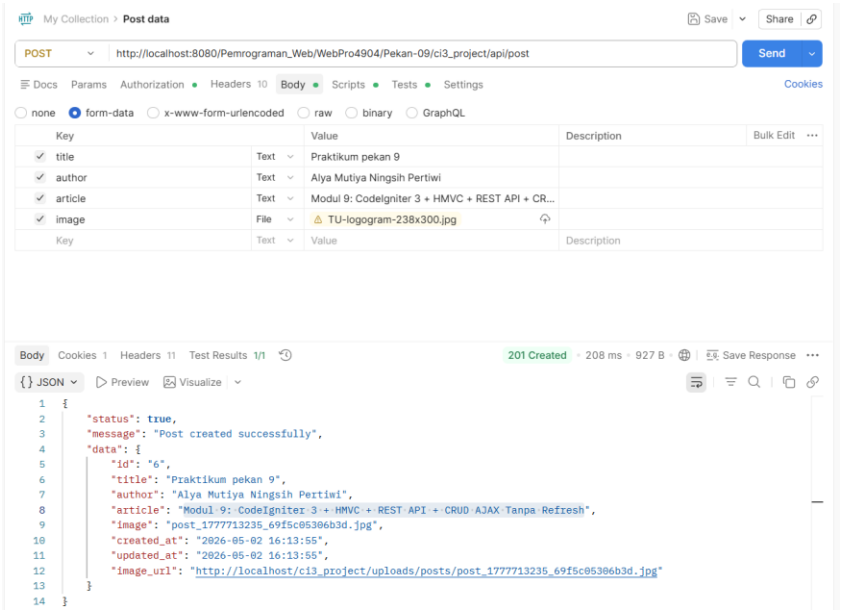
d. Detail akun



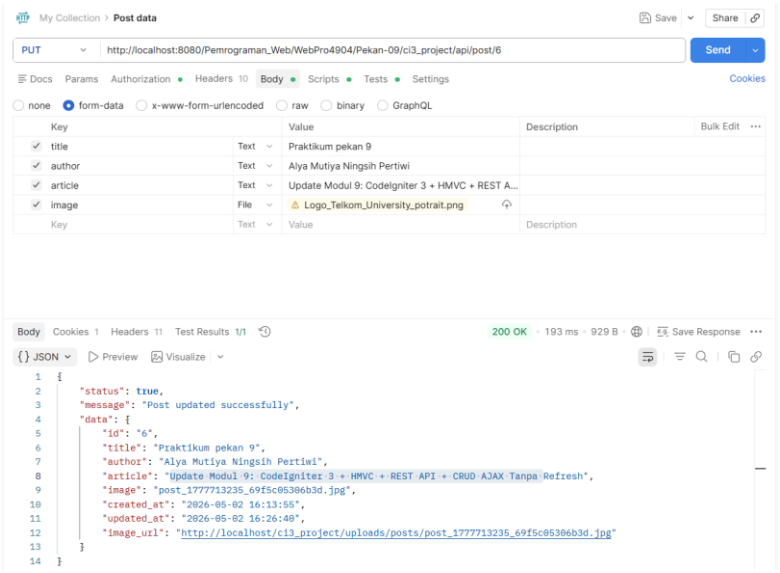
e. Logout



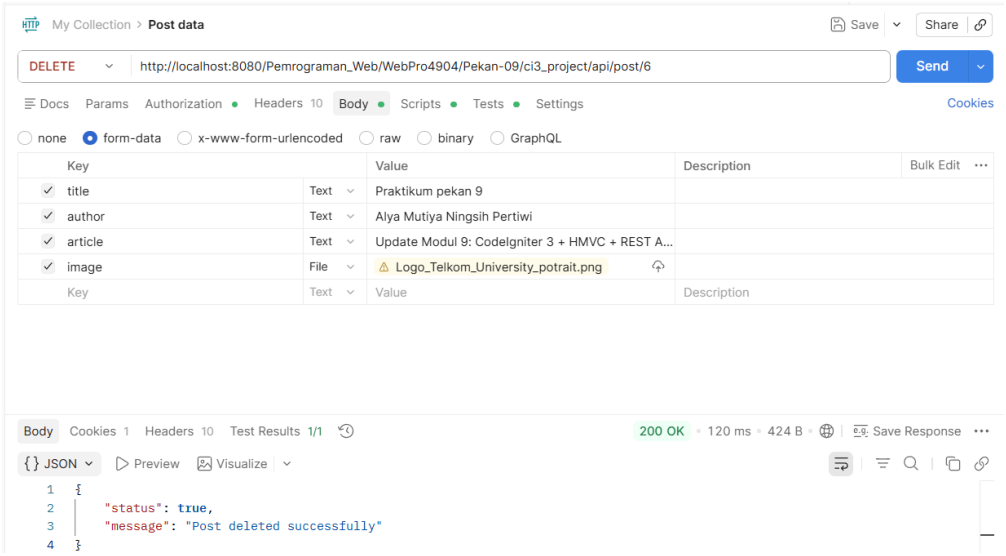
f. Create post



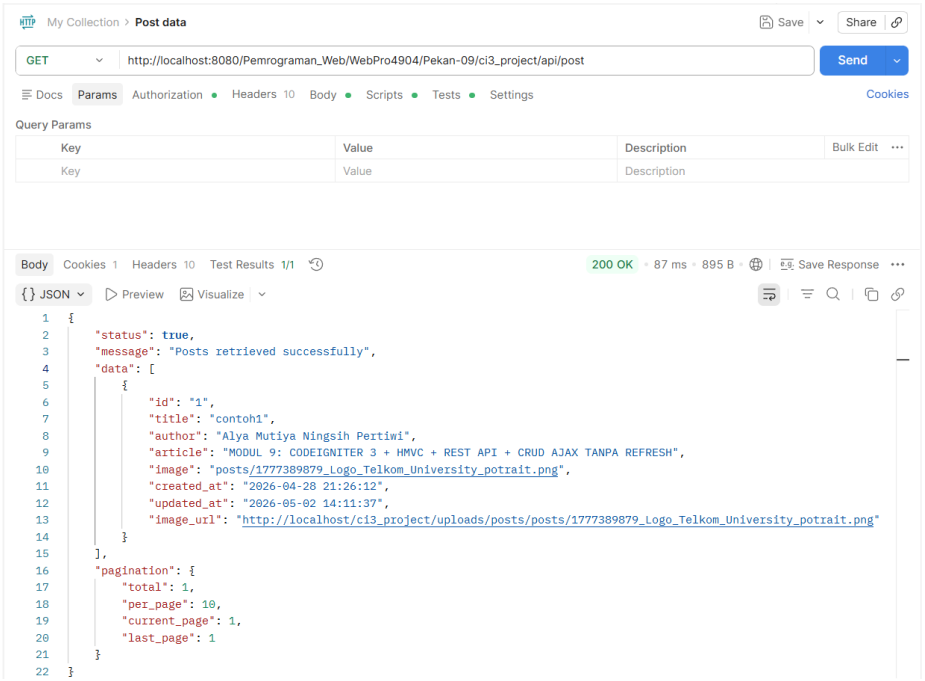
g. Update post



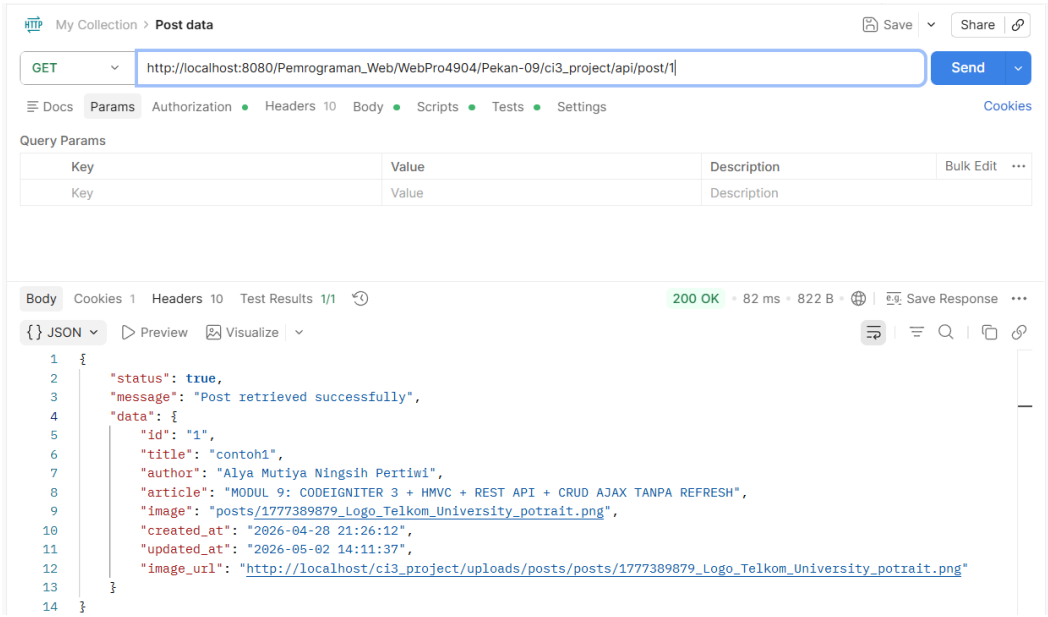
h. Delete post



i. Get all posts



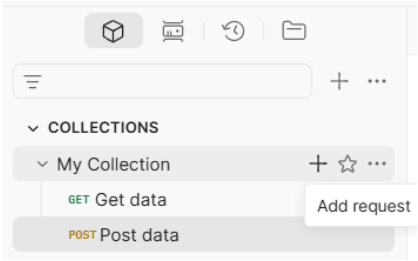
j. Get single post



Alur pengujian:

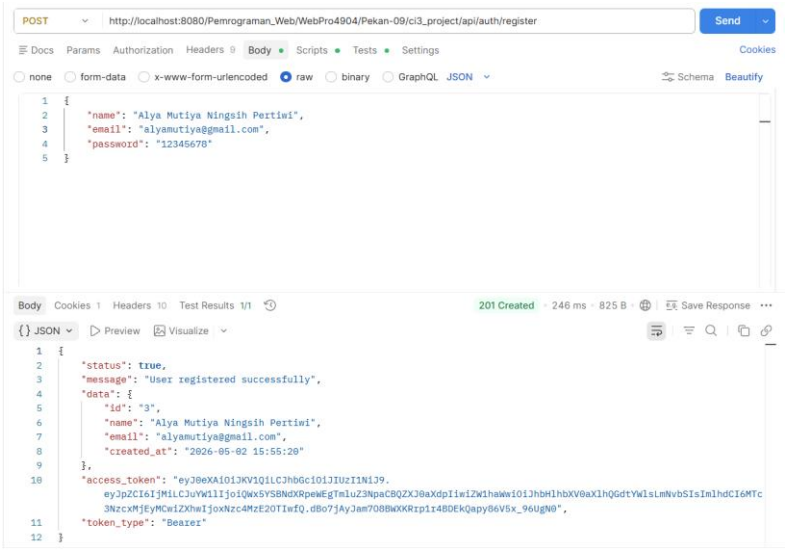
1. Buka aplikasi Postman

2. Kemudian klik Add Request pada bagian kiri atas pada My Collection.

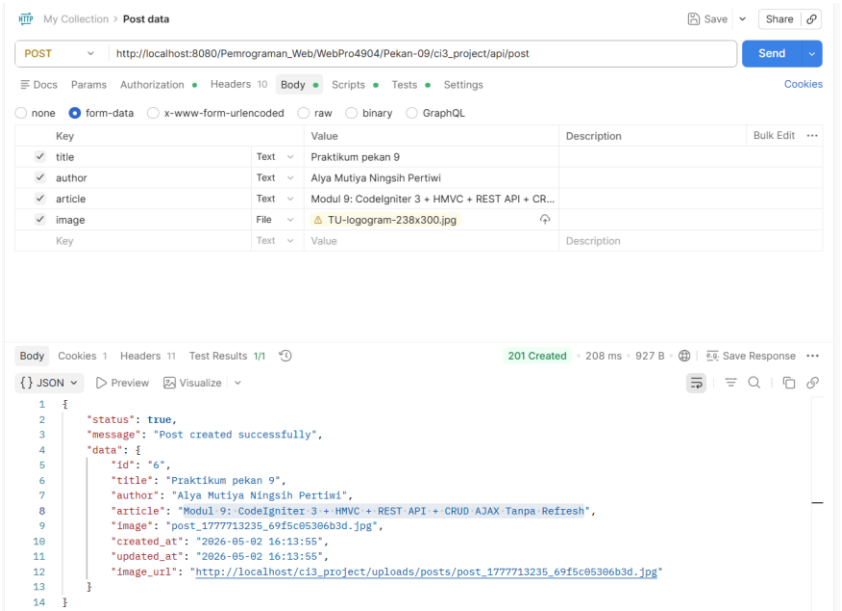


3. Kemudian lakukan perintah dan ketik alamat web pada bagian ini dan ubah menjadi POST jika ingin menambahkan data, ubah menjadi GET jika ingin menampilkan data yang telah di tambahkan, ubah menjadi PUT jika ingin melakukan update data, dan ubah menjadi DELETE jika ingin menghapus data yang telah di tambahkan, kemudian klik tombol 'Send' perintah dapat tersimpan.

   Contoh perintah:

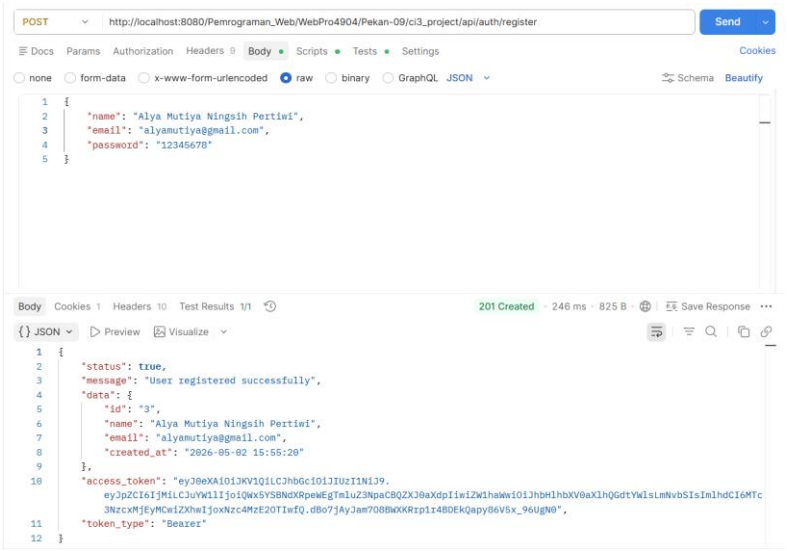


   Contoh perintah untuk menambah file dari komputer:

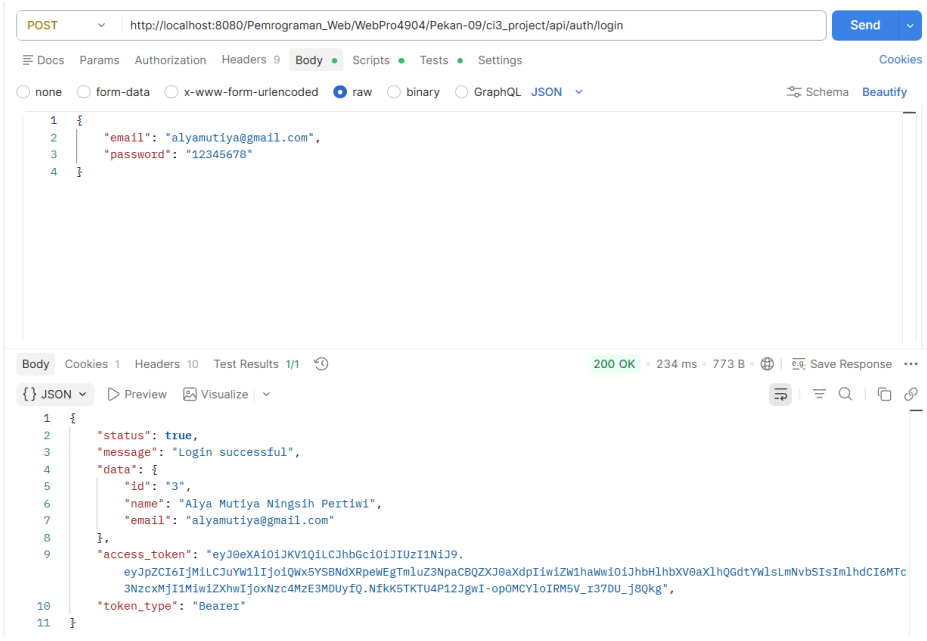


4. Hasil dan Dokumentasi Program

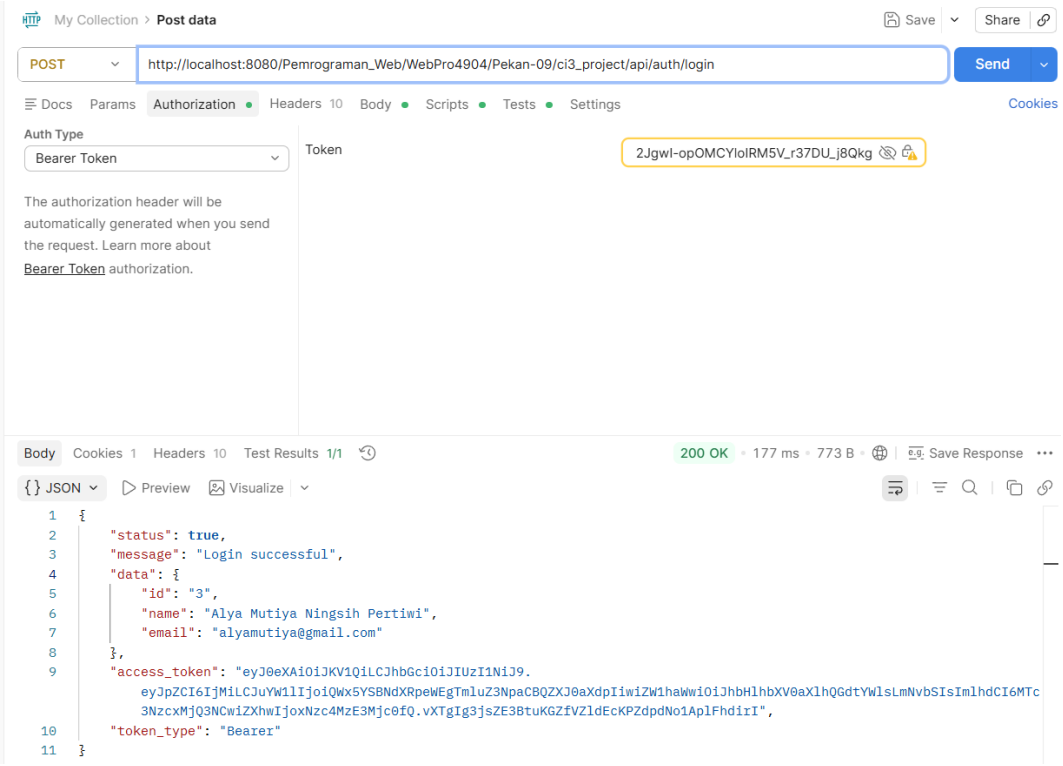
1. Register



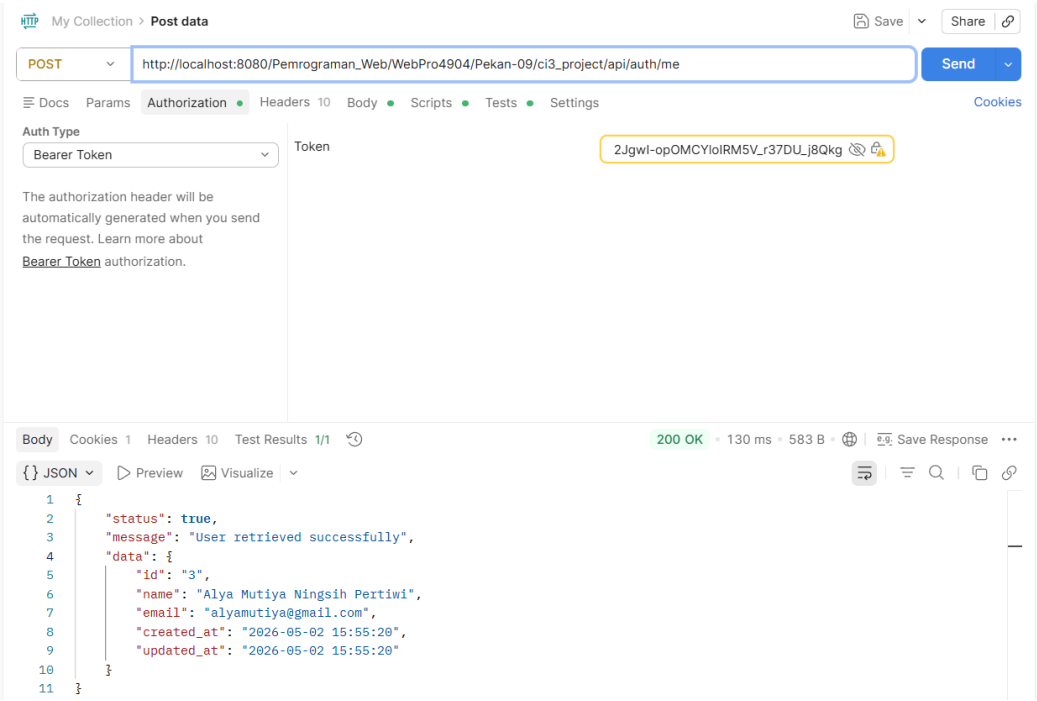
2. Login



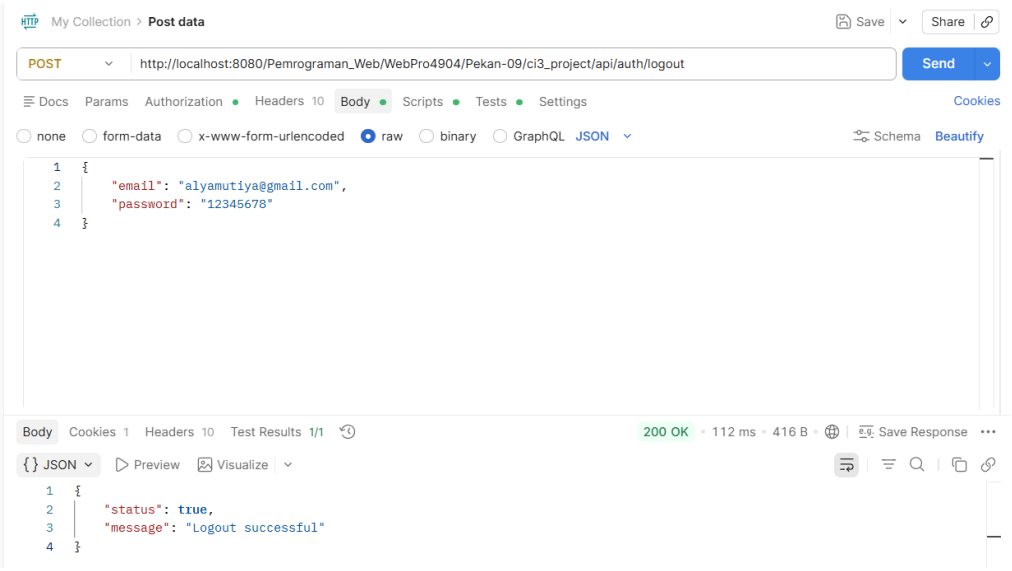
3. Token keamanan login



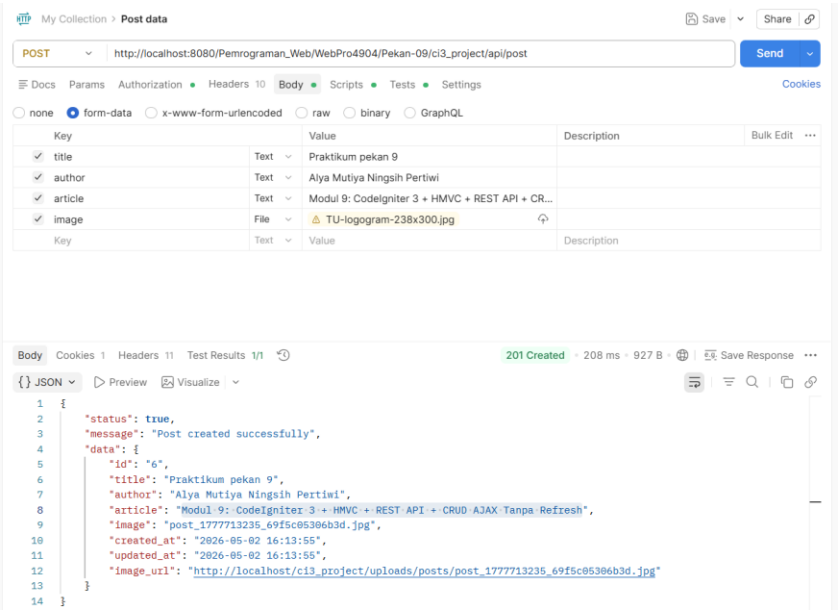
4. Detail akun



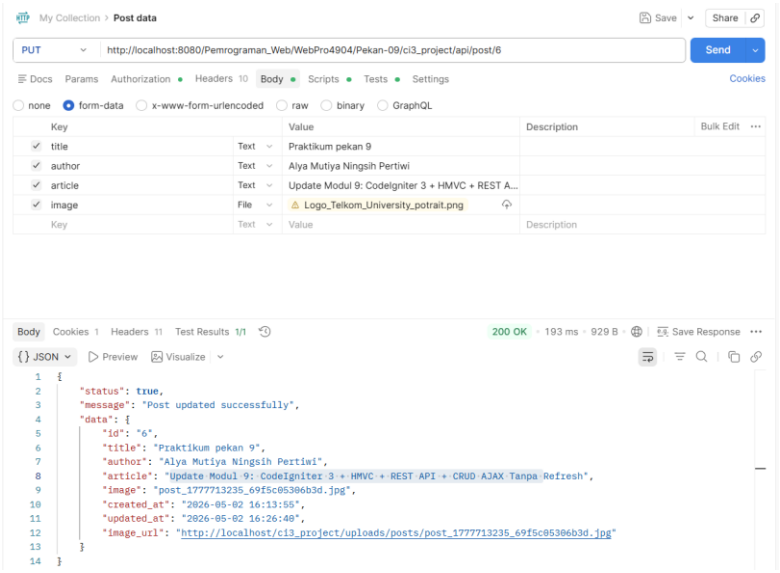
5. Logout



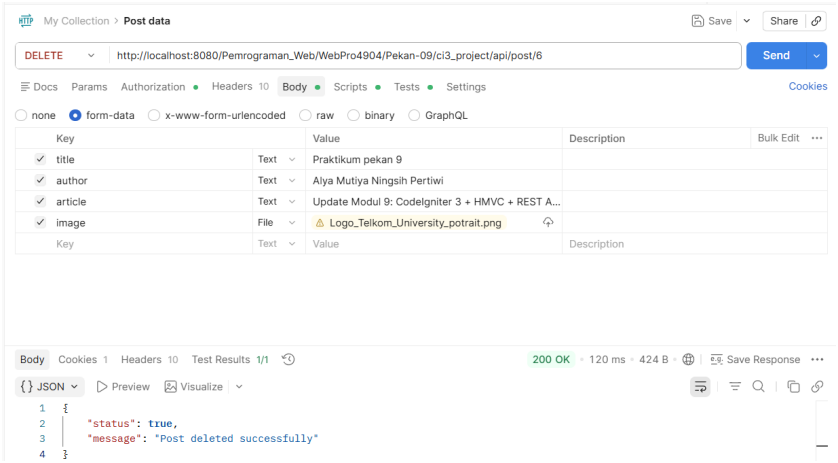
6. Create post



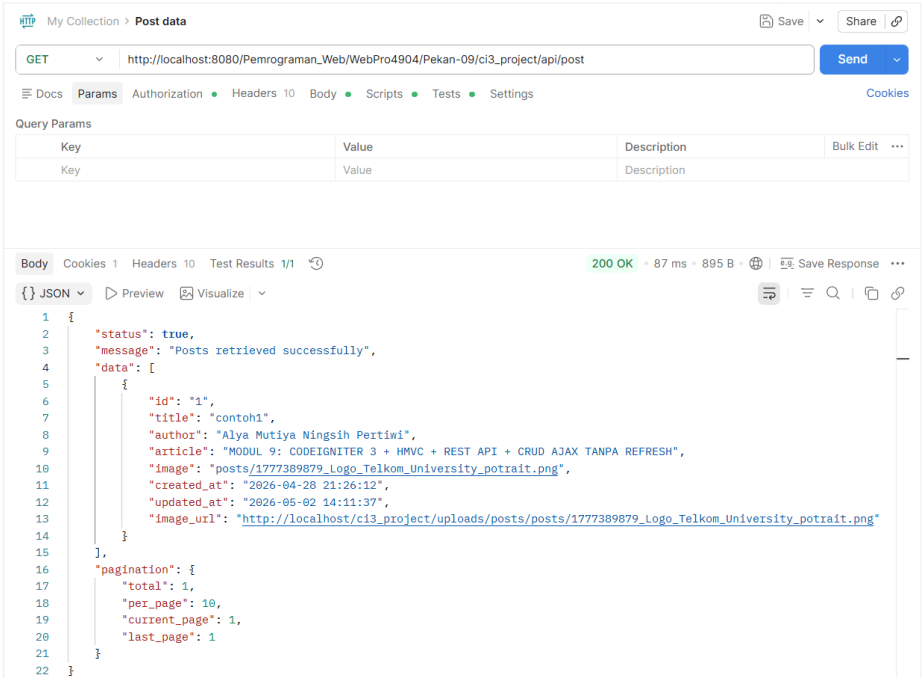
7. Update post



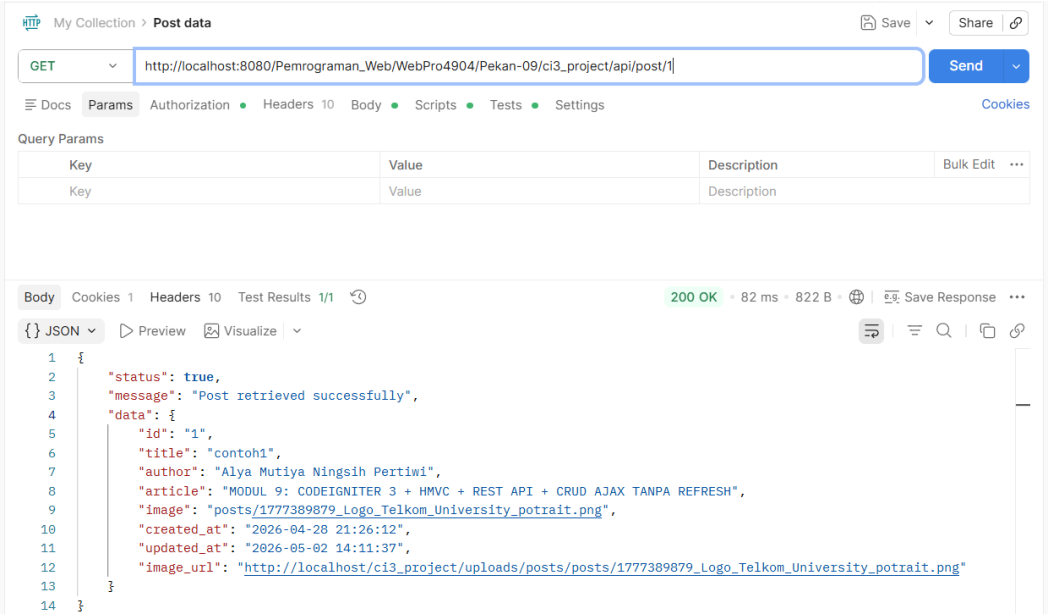
8. Delete post



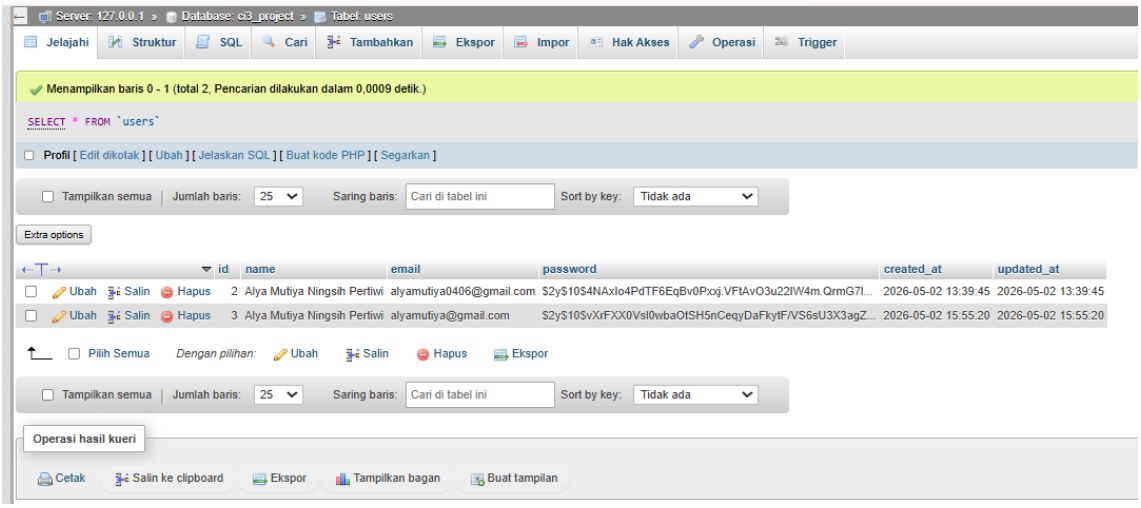
9. Get all posts



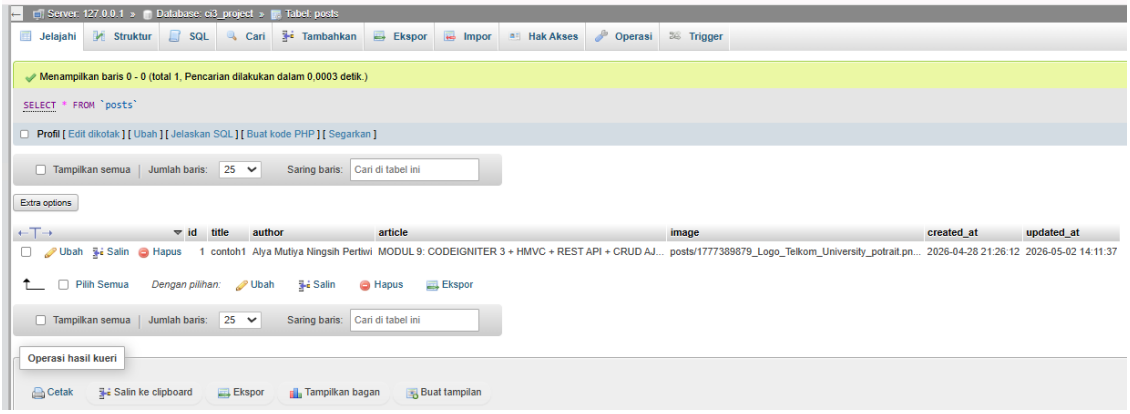
10. Get single post



11. Database user



12. Database post



5. Analisis dan Pembahasan

Pada praktikum Pekan 9 ini, aplikasi dikembangkan menggunakan CodeIgniter 3 dengan pendekatan HMVC, di mana setiap fitur dipisahkan ke dalam modul seperti

Crudjs dan Api. Struktur ini membuat kode lebih rapi, terorganisir, dan mudah dikembangkan.

Selain itu, diterapkan REST API untuk menghubungkan frontend dan backend menggunakan format JSON. Proses seperti login, pengambilan data, hingga CRUD dilakukan melalui metode HTTP (GET, POST, PUT, DELETE) dan dilengkapi dengan keamanan token.

Penggunaan AJAX memungkinkan proses CRUD berjalan tanpa reload halaman, sehingga aplikasi menjadi lebih responsif. Pengujian menggunakan Postman menunjukkan bahwa seluruh endpoint API berjalan dengan baik.

6. Kesimpulan

Praktikum ini menunjukkan bahwa penggunaan HMVC, REST API, dan AJAX pada CodeIgniter 3 mampu menghasilkan aplikasi yang lebih terstruktur, fleksibel, dan interaktif. Sistem menjadi lebih mudah dikembangkan serta memberikan pengalaman pengguna yang lebih baik.

7. Lampiran berisi Link repositori GitHub

https://github.com/AriqAhmadNayaka/WebPro4904/tree/AlyaMutiyaNingsihPertiwi-707012500132